Indexing in pandas

Strategies for selecting, modifying, and working with your data

Matthew Wright

Created : July, 2021

Last updated : September, 2021

Read more at https://wrighters.io

Table of Contents:

Chapter 1

Introduction

One of the most popular libraries in the Python ecosystem is pandas. It is described as "a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language". It is also quite hard to master, and many people get frustrated as they try to learn it. Some of this frustration comes from the huge scope of most pandas references. This short book focuses only on the topic of indexing and selecting data in pandas. It is not meant to be a comprehensive guide to all things pandas, but after reading it you will definitely understand how indexing works and be able to work with data in a pandas `DataFrame` or `Series`.

1.1 Getting started

To get the most out of this book, you should work through the examples in your own Python environment if the concept is not clear to you. Working through the examples on your own will allow you get more comfortable with syntax and exploring pandas on your own. You should run a recent version of pandas and NumPy. The examples in this book were written with the following versions of both libraries.

```
[1]: import pandas as pd
     pd.__version__
```

```
[1]: '1.3.2'
```

```
[2]: import numpy as np
     np.__version__
```

```
[2]: '1.21.1'
```

1.2 Recommended configuration

If you have never installed Python before or don't have a working environment, I'd recommend several options in increasing order of complexity and difficulty.

- Use Google colab. You can easily create a notebook and save it with a free Google account. In the default environment, pandas and NumPy are already installed. Other dependencies are easily installable there using the `%pip` magic. Just add the command to a cell and execute it, like this: `%pip install Faker`.
- Setup your own Python install using pyenv. You can read my guide to installing pyenv and creating virtual environments. I'd recommend using something like pyenv so that you can have any version of Python you want on your local machine and isolate it from the default versions. If you are using Mac OS X or Linux, the default Python that comes with the OS will most likely be old. You should also plan on using a virtual environment to allow you to install packages in the future, or to try different versions of pandas and Numpy.
- Install miniconda or anaconda, both downloadable from here. This is a good option for those familiar with conda, or those likely to venture into more complex code with multiple dependencies since conda can make managing that easier. It is not necessary to follow the examples in this book.

Once the environment is setup, you can follow along with the examples easily. Just install a recent version of pandas (and Numpy, which gets installed automatically), and if you want to work in a Jupyter notebook, install that as well. Here's what this would look like from the command line:

```
pip install "jupyter>=1.0.0" "pandas>=1.2.5 Faker"
```

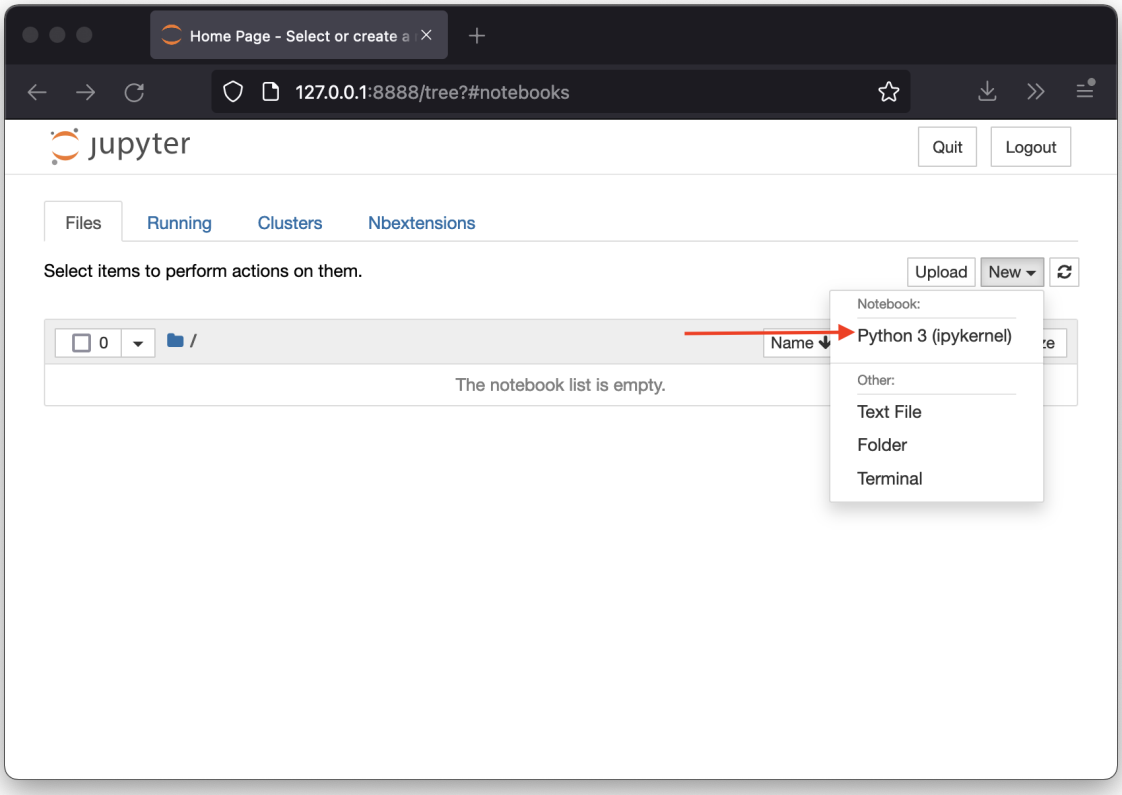
Or in a cell in a notebook:

```
%pip install "jupyter>=1.0.0" "pandas>=1.2.5 Faker"
```

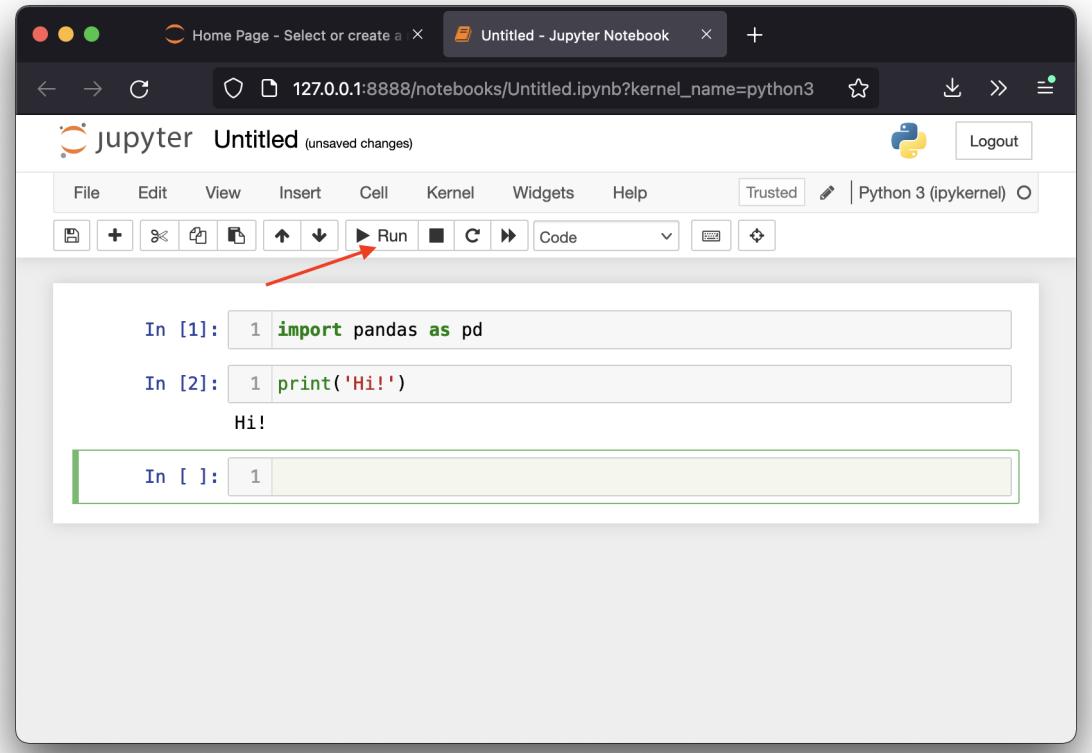
If you are going to work on your local machine, can now run the Jupyter notebook service from the command line. If you're using Google colab, you can just create a new notebook there. Also, if you're on your local machine, you may want to run this command from a directory that you create just for following along in this book.

```
jupyter notebook
```

Assuming things worked, you should have a browser window launched on your local machine. Click on the "New" menu and select the Python 3 option. It will look like this:



Then another tab will open, with a notebook. You enter code in the cells, and execute them by clicking the "Run" button, or Shift-Enter on your keyboard at the same time.



Now you have all you need to follow along with the rest of the book.

Let's get started!

Chapter 2

Understanding the basics

The topic of indexing and selecting data in pandas is core to using pandas, but it can be quite confusing. One reason for that is because over the years pandas has grown organically based on user requests so there are multiple way to select data out of a pandas `DataFrame` or `Series`. Reading through the documentation can be a real challenge, especially for beginners. For more advanced users, it is easy to learn a few techniques and just fall back on your favorite method to access data and not realize that there might be simpler, faster, or more reliable ways to both select and modify your data.

Since this can be a complicated and confusing topic, we will work our way up from basics to more advanced and complex scenarios. The methods used for selecting and indexing are some of the most confusing methods to work with in pandas due to their different behavior with different argument types. You should expect to be a little confused, but understanding the basics will make this much easier.

This first chapter is just going to cover selecting data by index label or integer offsets. In comming chapters, we'll discuss slicing, boolean indexing, the `query` method, time series indexing, and much more.

As we get started, we will want some data to work with. To keep things simple, we want to ensure that we have a dataset that contains the types of data necessary to understand these concepts. There is a nice library that will help us with this, called Faker.

Faker just creates realistic fake data. I'll use it throughout the book to make some data on the fly. Don't worry about how the data is created, just pay attention to what it looks like. What are the columns and their types? What is the index?

[1]:
```
import pandas as pd
import numpy as np

from faker import Faker
Faker.seed(0) # this ensures we make the same data every time we run this

fake = Faker()

size = 1000

df = pd.DataFrame([
    {'first_name': fake.first_name(),
     'last_name': fake.last_name(),
     'birthday': fake.date_between(pd.Timestamp('1940-01-01'), end_date=pd.Timestamp(
↪'2010-01-01')),
     'phone': fake.phone_number(),
     'id': f'X-{s:04d}'
    } for s in range(size)])
```

[2]:
```
df['birthday'] = pd.to_datetime(df['birthday']) # ensuring that this is a date type
```

[3]:
```
df.dtypes
```

[3]:
```
first_name            object
last_name             object
birthday      datetime64[ns]
phone                 object
id                    object
dtype: object
```

[4]:
```
df # let's see what we have...
```

[4]:
```
      first_name last_name   birthday               phone      id
0          Megan     Chang 1997-03-28          4876475938  X-0000
1         Donald    Taylor 1952-11-27  +1-489-241-1578x156  X-0001
2         Leslie    Bowman 1967-10-30     +1-778-408-0160  X-0002
3         Cheryl   Clayton 1985-05-04        513.933.2871  X-0003
4    Christopher    Flores 2009-02-16   148.418.5839x8947  X-0004
..           ...       ...        ...                 ...     ...
995      Timothy     Miles 1968-04-25          0424282878  X-0995
996      Michael    Levine 2008-09-10          2708030968  X-0996
997       Calvin       May 1948-11-14  229-069-6830x28734  X-0997
998       Andrew    Larson 1985-04-25   (489)935-7218x048  X-0998
999     Kimberly  Robinson 1999-11-30     +1-820-842-1676  X-0999

[1000 rows x 5 columns]
```

[5]:
```
df.columns  # what are the columns?
```

[5]:
```
Index(['first_name', 'last_name', 'birthday', 'phone', 'id'], dtype='object')
```

2.1 Axes: an index and (optionally) the columns

The two main data structures in pandas both have at least one axis. A `Series` has one axis, the index. A `DataFrame` has two axes, the index and the columns. It's useful to note here that in all the `DataFrame` functions that can be applied to either rows or columns, an axis of 0 refers to the index, an axis of 1 refers to the columns.

We can inspect these in our sample `DataFrame`. We'll pick the `landmark_name` column as a sample `Series` to demonstrate the basics for a `Series`. You can see the column (which is a `Series`) and the entire `DataFrame` share the same index.

[6]:
```
s = df['first_name']
print("Series index:", s.index)
print("DataFrame index:", df.index)
```

```
Series index: RangeIndex(start=0, stop=1000, step=1)
DataFrame index: RangeIndex(start=0, stop=1000, step=1)
```

2.2 Indexes are for lookups, data alignment, and reindexing

In pandas, an `Index` (or a subclass) allows for the data structures that use it to support lookups (or selection), data alignment (think of time-series data especially, where all the observations needs to be aligned with their observation time), and reindexing (changing the underlying index to have different values, but keeping the data aligned). There are a number of types of indices, but for now, we'll just look at the simple `RangeIndex` that our current `DataFrame` is using, which will have integer values.

2.3 Basic selecting with `[]`

We're going to start with the basic form of selecting, using the `[]` operator, which in Python maps to a class's `__getitem__` function (if you're familiar with objects in Python, if not, don't worry about that for now). Depending on whether the pandas object is a `Series` or a `DataFrame` and the arguments you pass into this function, you will get very different results. Let's start with the basics, invoking with a single argument.

2.3.1 Series

With a `Series`, the call will return a single scalar value that matches the value at that label in the index. If you pass in a value for a label that doesn't exist, you will get a `KeyError` raised. Also, if you pass in an integer and your index has that value, it will return it. But if you don't have an integer value in your index, it will return the value by position. This is convenient, but can be confusing.

Now I'm going to give this `DataFrame` (and `Series`) a new index because the `RangeIndex` could make much of the following examples very confusing. It's important for us to differentiate between accessing elements by label and by position. If our index labels are integers, you will not be able to see the difference! Since this dataset already has a unique `id` column, we'll use that instead.

[7]:
```
df.index = df['id']
s = df['first_name']
df.index
```

[7]:
```
Index(['X-0000', 'X-0001', 'X-0002', 'X-0003', 'X-0004', 'X-0005', 'X-0006',
       'X-0007', 'X-0008', 'X-0009',
       ...
       'X-0990', 'X-0991', 'X-0992', 'X-0993', 'X-0994', 'X-0995', 'X-0996',
       'X-0997', 'X-0998', 'X-0999'],
      dtype='object', name='id', length=1000)
```

Now that our index doesn't contain `int` values, when we call it with `int`s they will be evaluated as positional arguments. If your index had `int` values, they would be found first rather than posititional values. Confusing, isn't it? This is one reason why you want to read on and see why there are better ways to do this.

[8]:
```
print("The value for X-0014:", s['X-0014'])
print("The first value:", s[0])
print("The value for X-0017:", s['X-0017'])
print("The third value:", s[2])
try:
    s['L-900']
except KeyError as ke:
    print("Exception: ", ke)
```

```
The value for X-0014: Paul
The first value: Megan
The value for X-0017: Jennifer
The third value: Leslie
Exception:  'L-900'
```

While I rarely use it, there is a `get` method available, which will return `None` if the argument is not in the index instead of raising a `KeyError`.

[9]:
```
print("The first value:", s.get(0))
print("Is there a value at 'L-900'?: ", s.get('L-900'))
```

```
The first value: Megan
Is there a value at 'L-900'?:  None
```

2.3.2 DataFrame

Now with a `DataFrame`, calls to `[]` are used for selecting from the column index, not the row index. This can be confusing since it's different from a `Series` when passing in integer values. Instead, we pass in column names.

[10]:
```
try:
    print("First element in a Series:", s[0])
    print("First row in a DataFrame?:", df[0])
except KeyError as ke:
    print("Nope, that's not how you select rows in a DataFrame")

df['last_name']
```

```
First element in a Series: Megan
Nope, that's not how you select rows in a DataFrame
```

[10]:
```
id
X-0000       Chang
X-0001      Taylor
X-0002      Bowman
X-0003     Clayton
X-0004      Flores
            ...
X-0995       Miles
X-0996      Levine
X-0997         May
X-0998      Larson
X-0999    Robinson
Name: last_name, Length: 1000, dtype: object
```

We can also select a list of columns, in any order (even repeated).

[11]:
```
df[['first_name', 'phone', 'last_name']]
```

[11]:
```
         first_name                phone last_name
id
X-0000        Megan           4876475938     Chang
X-0001       Donald  +1-489-241-1578x156    Taylor
X-0002       Leslie      +1-778-408-0160    Bowman
X-0003       Cheryl         513.933.2871   Clayton
X-0004  Christopher    148.418.5839x8947    Flores
...             ...                  ...       ...
X-0995      Timothy           0424282878     Miles
X-0996      Michael           2708030968    Levine
X-0997       Calvin   229-069-6830x28734       May
X-0998       Andrew    (489)935-7218x048    Larson
X-0999     Kimberly      +1-820-842-1676  Robinson

[1000 rows x 3 columns]
```

2.4 Attribute access for columns on a DataFrame or values in a Series.

Another way to select a DataFrame column or an element in a Series is using the attribute operator, .. Pandas will "automagically" create accessors for all DataFrame columns or for all labels in a Series, provided their name translates to valid Python. This can make life easier for you, especially in an environment with tab-completion like IPython or a Jupyter notebook. In general, it's best not to use these attributes in production code. Why not? * Your column names may collide with method names on a DataFrame itself, and in that case you will be accessing something you weren't intending to. * Column names often may not be valid Python identifiers since they may contain spaces or start with (or just be) numbers, so you have to use the longer form anyway. * Using [] with quoted strings makes your code very clear for others (and future self) to read. * Assigning to a non-existing attribute won't create a new column, it will just create an attribute on the object. (We'll talk about modifying data in subsequent chapters)

So hopefully this list reinforces why it's just a bad habit to rely on attribute access. In our data example, we can use attribute access for some of our column names, but because the primary index doesn't make valid Python identifiers, we can't use it on our Series (L-265 is not a valid Python identifier, for example). But it does work for some situations and may save you a few keystrokes when doing exploratory analysis.

[12]:
```
df.first_name
```

[12]:
```
id
X-0000          Megan
X-0001         Donald
X-0002         Leslie
X-0003         Cheryl
X-0004    Christopher
              ...
X-0995        Timothy
X-0996        Michael
X-0997         Calvin
X-0998         Andrew
X-0999       Kimberly
Name: first_name, Length: 1000, dtype: object
```

2.5 Selecting with .loc using labels

Pandas also has the .loc indexer which is intended for selection and indexing by label. This is similar to [] on a Series. However, using .loc and .iloc will make it more clear in your code what your intentions are, and they behave differently. Note that .loc will raise KeyError when an element doesn't exist at that label.

Also note that .loc is not a method, so we don't call it, but rather we use the indexing operator ([]) on it.

2.5.1 Series

Let's start with a Series. First, note that we don't select by location, we select by index label.

[13]:
```
try:
    print("Selecting by location?", s.loc[139])
except KeyError as ke:
    print("Nope, not with .loc: ", ke)

print("Yes, do it by label: ", s.loc['X-0139'])
```

```
Nope, not with .loc:  139
Yes, do it by label:  Robert
```

We can also pass in a list of labels, and they will be returned as a `Series`.

[14]:
```
s.loc[['X-0012', 'X-0265', 'X-0224']]
```

[14]:
```
id
X-0012      Alyssa
X-0265       Jesse
X-0224    Courtney
Name: first_name, dtype: object
```

Note that this can be a list with a single element, but passing in a list returns a `Series` (with one element), not a scalar.

[15]:
```
s.loc[['X-0012']]
```

[15]:
```
id
X-0012    Alyssa
Name: first_name, dtype: object
```

2.5.2 DataFrame

On a `DataFrame`, a single argument to `.loc` will return a `Series` for the row matching the label.

[16]:
```
df.loc['X-0011']
```

[16]:
```
first_name                Candace
last_name                   Lyons
birthday      1993-04-02 00:00:00
phone                 457.923.0225
id                          X-0011
Name: X-0011, dtype: object
```

If passed a list of labels, `.loc` will return a `DataFrame` of the matching rows. This is just selecting rows by index, but selecting multiple rows. Note that all of the elements in the list have to be in the index or `KeyError` is raised.

[17]:
```
df.loc[['X-0012', 'X-0011', 'X-0013']]
```

[17]:
```
       first_name last_name   birthday                  phone      id
id
X-0012     Alyssa    Wilson 2000-03-13  +1-207-698-4564x28071  X-0012
X-0011    Candace     Lyons 1993-04-02           457.923.0225  X-0011
X-0013    Matthew  Williams 1978-03-24          (375)945-9924  X-0013
```

However, `DataFrame`'s `.loc` can take multiple arguments. The first argument is an indexer for *rows*, the second argument is an indexer for *columns*. So if we wanted a row and the `phone` column, we can get back a scalar.

[18]:
```
df.loc['X-0012','phone']
```

[18]:
```
'+1-207-698-4564x28071'
```

Now let's move on from `.loc` for now, but circle back later to talk about some more advanced ways of selecting data with it.

2.6 Selecting with `.iloc` using integer offsets

`.iloc` is a separate method for use with purely integer based indexing, starting from 0. It's very similar to the behavior of `.loc`, as we'll see, but raises `IndexError` when the indexer is out of bounds, or the wrong type.

2.6.1 Series

For a `Series`, `.iloc` returns a scalar, just like `.loc`.

[19]:
```
s.iloc[0]
```

[19]:
```
'Megan'
```

Relative indexing is allowed, and if you try to access a non-existent element, `IndexError` is raised.

[20]:
```
s.iloc[-1]
```

[20]:
```
'Kimberly'
```

[21]:
```
try:
    s.iloc[9999]
except IndexError as ix:
    print("You ran over the end of your Series: ", ix)
```

```
You ran over the end of your Series:  single positional indexer is out-of-bounds
```

Passing in a list returns a `Series`. Again, all elements in the list need to be in the index, or `IndexError` is raised.

[22]:
```
s.iloc[[0,1,2]]
```

[22]:
```
id
X-0000     Megan
X-0001    Donald
X-0002    Leslie
Name: first_name, dtype: object
```

2.7 What is this `.ix` indexer I see in old code?

Pandas used to have another indexer, `.ix`. This indexer was meant to primarily work as a label based indexer, with a fallback to integer positioning. Think of it as `.loc` with a fallback to `.iloc` if the first one didn't work. It has been deprecated starting in pandas 0.20.0, and is not available in newer versions of pandas. If you see it used in older code, you should try converting that call to use `.loc`, unless it's clear that it's meant to use integer indexing. In that case, use `.iloc`.

2.8 Index exact elements with `.at` or `.iat`

Sometimes you want to only access a single value in a `Series` or `DataFrame`. We've already seen how you can do this in a `Series` using using the other indexers by passing in a single value. The intention of `.at` and `.iat` are just for single labels or offsets and return a single value. In other words, they really are just easier to remember, you can get the same results with `.loc` and `.iloc`. We use labels for `.at`. In a `DataFrame`, supply the column as the second argument.

[23]:
```
print('By label in a Series: ', s.at['X-0000'])
print('By label in a DataFrame: ', df.at['X-0000', 'first_name'])
```

```
By label in a Series:  Megan
By label in a DataFrame:  Megan
```

We use integer offsets for `.iat`.

[24]:
```
print('By integer offset in a Series: ', s.iat[0])
print('By integer offset in a DataFrame: ', df.iat[0, 0])
```

```
By integer offset in a Series:  Megan
By integer offset in a DataFrame:  Megan
```

With just this brief introduction, you now understand enough about `[]`, `.loc`, and `.iloc` to know the difference between the three methods. This is enough to be quite effective, but we can do so much more.

Chapter 3

Slicing data in pandas

This chapter is a review of slicing - a core Python topic, but with a few subtle issues related to pandas. After covering slicing, we'll review the selection methods from the last chapter, but now with slicing applied.

3.1 What is a slice, and what is slicing?

A slice is defined in the Python glossory.

> An object usually containing a portion of a sequence. A slice is created using the subscript notation, [] with colons between numbers when several are given, such as in variable_name[1:3:5]. The bracket (subscript) notation uses slice objects internally.

Slicing is defined in the Python reference.

> a slicing selects a range of items in a sequence object (e.g., a string, tuple or list). Slicings may be used as expressions or as targets in assignment or del statements.

A slicing has three parts: start, stop, and step. A `slice` class is a Python builtin. If you instantiante a `slice` object explicitly, you can supply a single value (the stop), or you can supply the start and stop and optionally, the step. In terms of what is selected in the sequence, the start is inclusive, the stop is not, and the step determines how you count from start to stop. You can use slice objects or slice notation, but unless you need to store the slice object for re-use, you usually see slice notation in code.

Here are a few examples to clear things up. To keep things simple, take the example of a string as a sequence object. Strings can be sliced in Python, just like an array of numbers would be.

[1]:
```
w = "abcdefg"

# all of these select the same slice of w, "abc"

w[slice(0, 3, 1)]  # This is slice(start=0, stop=3, step=1)
w[0:3:1]           # This is the same, but in slice notation
w[slice(3)]        # start=0, step=1 are the defaults
w[:3]              # same here

# select the entire sequence. start=0, stop=last element (inclusive), step=1
w[:]
```

[1]:
```
'abcdefg'
```

Now slices can be used very effectively and concisely to get views of data. Here's a few examples.

[2]:
```
w[::-1] # reverse a string (start=0, stop=end, stepping backwards)
```
[2]:
```
'gfedcba'
```

[3]:
```
w[1:3] # a portion of a string
```
[3]:
```
'bc'
```

Now strings are special, you can't modify them directly or delete their values. The same goes for tuples.

[4]:
```
try:
    w[2] = 'z'
except TypeError as te:
    print(te)

try:
    t = (0, 1, 2)
    t[0] = -1
except TypeError as te:
    print(te)
```
```
'str' object does not support item assignment
'tuple' object does not support item assignment
```

If we switch to a list, we can view a more more applications of slicing, including assignment.

[5]:
```
l = [0, 1, 2, 3, 4, 5, 6]

# just as with strings, these all work on a list, and return [0, 1, 2]
l[slice(0,3,1)]
l[0:3:1]
l[slice(3)]
l[:3]

# this returns a full copy
l[:]
```
[5]:
```
[0, 1, 2, 3, 4, 5, 6]
```

[6]:
```
# using step
l[::2]
```
[6]:
```
[0, 2, 4, 6]
```

[7]:
```
# can also assign
l[1::2] = [-1, -1, -1]
l
```
[7]:
```
[0, -1, 2, -1, 4, -1, 6]
```

OK, after a quick overview of slicing, let's move onto pandas and see why we're talking about slicing at all.

Just like in the previous chapter, we will create a fake dataset.

[8]:
```
import pandas as pd
import numpy as np

from faker import Faker
Faker.seed(0)
```

(continues on next page)

(continued from previous page)

```
fake = Faker()

size = 1000
df = pd.DataFrame([
    {'company_name': fake.company(),
     'phone': fake.phone_number(),
     'website': fake.url(),
     'id': f"X-{s:04d}"
    } for s in range(size)])
```

[9]:
```
df.dtypes
```

[9]:
```
company_name    object
phone           object
website         object
id              object
dtype: object
```

3.2 Slicing in pandas

Now in pandas (and the underlying NumPy data structures), slicing is also supported, and you can use it via the indexing operator (`[]`), `.loc`, and `.iloc`. Just like in the last chapter, we'll walk through these three methods and look at slicing for both `Series` and `DataFrame` objects. Also, as last time, to prevent confusion between location based indexing and label indexing, I'm going to update the index of our sample data to be a string for the initial explanations.

[10]:
```
# this will make our examples a bit more obvious
df = df.set_index('id', drop=True)
# Use the company_name column for a sample Series, it will have the same index.
s = df['company_name']
```

[11]:
```
df.head(3)
```

[11]:

id	company_name	phone	website
X-0000	Chang-Fisher	876-475-9382x421	https://www.garcia-jones.com/
X-0001	Castro-Gomez	938-778-4080x16097	https://www.nguyen.org/
X-0002	Allen and Sons	+1-711-587-1484	http://harris.com/

3.3 Slicing with []

Just like basic indexing, we'll see that slicing with `.loc` and `.iloc` is preferred to using `[]`. Since the behavior of `[]` can depend on the index and arugments, it's better to use the more explicit indexers. But it is supported, there's a just a few things to keep in mind.

3.3.1 Series

Just like standard Python, you can use integer slicing on Series objects, with `start:stop:step`.

[12]:
```
s[:3]            # These all slice the first 3 elements in the Series
s[0:3]
s[0:3:1]
s[slice(0,3,1)]
```

[12]:
```
id
X-0000      Chang-Fisher
X-0001      Castro-Gomez
X-0002    Allen and Sons
Name: company_name, dtype: object
```

In a `Series`, we also can slice on our index, which in this case is an index of objects (strings). Note that the slice start and stop are inclusive, which is different than regular Python slicing!

[13]:
```
s['X-0004':'X-0009']
```

[13]:
```
id
X-0004    Lawrence, Herring and Riley
X-0005                    Logan Ltd
X-0006      Smith, Woodward and Lopez
X-0007                  Carlson Ltd
X-0008        Lowery, Banks and Price
X-0009                 Goodman-Poole
Name: company_name, dtype: object
```

You can specify a step as well.

[14]:
```
s['X-0010':'X-0014':2]
```

[14]:
```
id
X-0010          Griffin LLC
X-0012    Callahan and Sons
X-0014       Woods and Sons
Name: company_name, dtype: object
```

In our case, we have an index that is sorted, so if we search for elements that are *not* in the index, it will still work. It will just return an empty `Series` if no elements were found between the start and end points.

[15]:
```
s['ID-10':'ID-999']
```

[15]:
```
Series([], Name: company_name, dtype: object)
```

However, note that this is only working because our index is sorted. At the end of this chapter, we'll see how this doesn't work with unsorted indexes.

You can also (potentially) assign/update a slice. Note that using `[]` is not the right way to do this, you'll get a `SettingWithCopyWarning`. There will be more on this in future chapters.

[16]:
```
s['X-0002':'X-0004'] = "Changed"
s.head(6)
```

[16]:
```
id
X-0000    Chang-Fisher
X-0001    Castro-Gomez
X-0002         Changed
X-0003         Changed
X-0004         Changed
X-0005       Logan Ltd
Name: company_name, dtype: object
```

3.3.2 DataFrame

In a `DataFrame`, slicing with `[]` will **slice on the rows**. If you remember back to the last chapter, selection on a `DataFrame` using `[]` selects the columns first, so this is a bit confusing. Just remember, if you see a `:`, it's slicing on rows.

[17]:
```
df[1:3]
```

[17]:
```
        company_name               phone                   website
id
X-0001  Castro-Gomez  938-778-4080x16097  https://www.nguyen.org/
X-0002       Changed     +1-711-587-1484       http://harris.com/
```

You can also use the step.

[18]:
```
df['X-0003':'X-0007':2]
```

[18]:
```
       company_name               phone                website
id
X-0003      Changed  934-232-0947x11220     http://warren.com/
X-0005    Logan Ltd         098.910.1399       http://wood.com/
X-0007  Carlson Ltd  (345)792-3022x5841  https://www.ruiz.org/
```

3.4 Slicing with `.loc`.

Now, just like with basic selection, `.loc` and `.iloc` are meant for label and position respectively. With `.loc` you cannot pass in locations unless those match the index you are using. In our case, we'll get a `TypeError`. The issue is our index doesn't contain these values, and so we'll be rejected.

Note that with `.loc`, we will get both the start and stop values if they are in the index, just like with `[]`.

3.4.1 Series

For our example data, you can't slice with integers (since they aren't in the index), but we can slice by labels.

[19]:
```
try:
    s.loc[1:3]
except TypeError as te:
    print("TypeError", te)

s.loc['X-0001':'X-0003']
```

```
TypeError cannot do slice indexing on Index with these indexers [1] of type int
```

[19]:
```
id
X-0001    Castro-Gomez
```

(continues on next page)

(continued from previous page)

```
X-0002         Changed
X-0003         Changed
Name: company_name, dtype: object
```

Assignment or updates using slices (can) work with `.loc`. We'll talk about updates in more detail later chapters.

[20]:
```
s.loc['X-0004':'X-0005'] = "Changed again"
s.head(6)
```

[20]:
```
id
X-0000     Chang-Fisher
X-0001     Castro-Gomez
X-0002          Changed
X-0003          Changed
X-0004    Changed again
X-0005    Changed again
Name: company_name, dtype: object
```

3.4.2 DataFrame

And things are similar for a `DataFrame`.

[21]:
```
try:
    df.loc[1:3]
except TypeError as te:
    print(te)

df.loc['X-0001':'X-0003']
```

```
cannot do slice indexing on Index with these indexers [1] of type int
```

[21]:
```
        company_name               phone                   website
id
X-0001  Castro-Gomez  938-778-4080x16097  https://www.nguyen.org/
X-0002       Changed     +1-711-587-1484       http://harris.com/
X-0003       Changed  934-232-0947x11220       http://warren.com/
```

Now with a `DataFrame`, you can slice both on the index and the columns, by label, with a step.

[22]:
```
df.loc['X-0010':'X-0020':2, "company_name":"phone"]
```

[22]:
```
                        company_name                 phone
id
X-0010                   Griffin LLC  001-278-789-0075x470
X-0012             Callahan and Sons  +1-851-240-0034x8559
X-0014                Woods and Sons            4281465461
X-0016                   Mora-Kramer  001-361-534-9263x511
X-0018                   Reid-Baxter       +1-930-555-0824
X-0020  Simmons, Fischer and Roberts   +1-676-993-0024x894
```

I'll also point out a common idiom in pandas code to use a full slice (`:`) to select the entire object when not using slicing to limit selection.

[23]:
```
df['company_name']           # this is one way to select a single column, for example
df.loc[:,'company_name']     # but this is how you select that column using .loc
```

[23]:
```
id
X-0000                    Chang-Fisher
```

(continues on next page)

(continued from previous page)

```
X-0001                     Castro-Gomez
X-0002                          Changed
X-0003                          Changed
X-0004                    Changed again
                     ...
X-0995                     Taylor-Campos
X-0996                  Alexander-Hughes
X-0997                      Thomas Group
X-0998                      Gonzalez Inc
X-0999    Robinson, Acosta and Matthews
Name: company_name, Length: 1000, dtype: object
```

3.5 Slicing with `.iloc`

Remember, `.iloc` is for strictly integer based indexing, so as you'd expect, it doesn't work with labels. But also note that `.iloc` works like standard Python slicing, i.e. the stop value is not included.

3.5.1 Series

[24]:
```
try:
    s.iloc["X-0001":"X-0003"]  # not this way with .iloc
except TypeError as te:
    print(te)
s.iloc[1:3]                # this way
```

```
cannot do positional indexing on Index with these indexers [X-0001] of type str
```

[24]:
```
id
X-0001    Castro-Gomez
X-0002         Changed
Name: company_name, dtype: object
```

3.5.2 DataFrame

Similarly, `DataFrame` selects from the index as the first argument, and the columns with the second argument.

[25]:
```
df.iloc[1:3]
```

[25]:
```
        company_name               phone                   website
id
X-0001  Castro-Gomez  938-778-4080x16097  https://www.nguyen.org/
X-0002       Changed     +1-711-587-1484       http://harris.com/
```

[26]:
```
df.iloc[1:3, 0:2]
```

[26]:
```
        company_name               phone
id
X-0001  Castro-Gomez  938-778-4080x16097
X-0002       Changed     +1-711-587-1484
```

[27]:
```
df.iloc[1:10:3, 2:-2]
```

[27]:
```
Empty DataFrame
Columns: []
Index: [X-0001, X-0004, X-0007]
```

3.6 Some special cases for slicing

There's always some special cases to consider, so I'll go through those last.

3.6.1 Changing behavior for integer based indexes

You'll remember that I changed the index on our first test `DataFrame` to be a string based index instead of the standard `RangeIndex` of sequential integer values. This following example shows why, but you should now understand why it works this way. You might think that passing in the same exact arguments to both `.loc` and `.iloc` would yield the same results, but now you realize that's not true.

[28]:
```
df2 = pd.DataFrame({"a": [0, 1, 2, 3], "b": [4, 5, 6, 7], "c": [8, 9, 10, 11]})
df2
```

[28]:

	a	b	c
0	0	4	8
1	1	5	9
2	2	6	10
3	3	7	11

[29]:
```
df2.loc[0:2]  # Remember, .loc is label based, and is inclusive of the endpoints
```

[29]:

	a	b	c
0	0	4	8
1	1	5	9
2	2	6	10

[30]:
```
df2.iloc[0:2]  # And .iloc is position based, and is not inclusive of the second␣
↪endpoint
```

[30]:

	a	b	c
0	0	4	8
1	1	5	9

[31]:
```
df2[0:2]   # So here [] behaves like iloc (position based), even though we may be␣
↪trying to use our index
```

[31]:

	a	b	c
0	0	4	8
1	1	5	9

[32]:
```
df2.index = ["w", "x", "y", "z"] # replace the index, now it's not integer based
df2["w":"y"]                     # now, [] behaves like it's label based.
```

[32]:

	a	b	c
w	0	4	8
x	1	5	9
y	2	6	10

Hopefully this example reinforces for you why you'll want to use `.loc` and `.iloc` in production code, especially when you're taking values to select rows or columns from unknown sources! Having it behave in strange ways based on the type of the index can make for very subtle bugs or just plain confusion.

3.6.2 Label slicing and sorted indexes

We saw that using `.loc` will include both the start and stop values of a slice, which is not the way standard Python works. Well, you can end up with indexes that are not sorted and will behave slightly differently than expected. Essentially, what you need to notice is that with a sorted index, all values in between the start and stop will be included (including the value for the stop label)

[33]:
```
s2 = pd.Series([1, 2, 3, 4, 5], index=[5, 2, 1, 9, 7])
s3 = pd.Series([1, 2, 3, 4, 5], index=[0, 1, 2, 3, 4])

s3.loc[0:3] # works just as you'd expect for .loc (not standard python slicing,␣
↪includes end label)
```

[33]:
```
0    1
1    2
2    3
3    4
dtype: int64
```

[34]:
```
s3.loc[0:9]  # no exception raised, even though 9 is not in the index, and includes␣
↪all values between them
```

[34]:
```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

[35]:
```
try:
    s2.loc[0:9]   # hmmm, this won't work for s2, unsorted index
except KeyError as ke:
    print(ke)
```

```
0
```

[36]:
```
s2.sort_index().loc[0:9] # this works though.
```

[36]:
```
1    3
2    2
5    1
7    5
9    4
dtype: int64
```

The reason for this functionality is that it is too expensive to do the sorting first, and it may not return the values you'd expect anyway since sorting may not make sense for your index. Just be aware of this behavior when you are dealing with non-sorted indexes. You'll realize that it's important to understand the indexes on your data and ensure they are accurately reflecting your data and what you're trying to accomplish with indexing and slicing it.

3.6.3 The Ellipsis and NumPy slicing

This is a little known part of Python, but in working through these examples I did come across the `Ellipsis` keyword in looking at the slicing documentation. `Ellipsis` is just a singleton (like `None`) and is intended to be used in extended slicing syntax by user-defined container data types. It is represented by the ellipsis literal, `...`.

It doesn't appear to be supported much by pandas beyond `Series`, but it is more useful in NumPy. The way to think of it is as a special value that will insert as many full slices as needed to extend the slice at the point of insertion in an index in all directions. This makes more sense with an example.

[37]:
```
...   # the singleton
```

[37]:
```
Ellipsis
```

[38]:
```
m = np.arange(27).reshape((3,3,3))  # 3 x 3 x 3 multi-dimensional array
print("Full array:\n", m)
print("First element in last dimension:\n", m[:,:,0])
print("With ellipsis:\n", m[...,0])
print("Last element in last dimension:\n", m[...,-1])
```

```
Full array:
 [[[ 0  1  2]
  [ 3  4  5]
  [ 6  7  8]]

 [[ 9 10 11]
  [12 13 14]
  [15 16 17]]

 [[18 19 20]
  [21 22 23]
  [24 25 26]]]
First element in last dimension:
 [[ 0  3  6]
 [ 9 12 15]
 [18 21 24]]
With ellipsis:
 [[ 0  3  6]
 [ 9 12 15]
 [18 21 24]]
Last element in last dimension:
 [[ 2  5  8]
 [11 14 17]
 [20 23 26]]
```

This could come in handy if you end up dealing with highly dimensioned arrays and want to save some typing. Or if you just want to try to stump someone with some obscure Python knowledge.

3.6.4 Wrapping it up

OK, that should be enough slicing to make you slightly dangerous. Hopefully you've learned something about slicing that you didn't know already.

In the next chapter we'll move on to boolean indexing, a very powerful way to select any bit of data in your `Series` or `DataFrame` using any logic you can think of.

Chapter 4

Boolean indexing

Using boolean indexing is probably the most common way to select data out of a `DataFrame` or `Series`, so this chapter contains the most useful information for a beginner to effectively use pandas for real data exploration and manipulation.

4.1 Boolean indexing uses a boolean vector to select values in the index

For those familiar with NumPy, this may already be second nature, but for beginners it is not so obvious. Boolean indexing works for a given array by passing a boolean vector into the indexing operator (`[]`), returning all values that are `True`.

One thing to note about boolean indexing, the selection array needs to be the same length as the array dimension being indexed.

Let's look at an example.

[1]:
```
import pandas as pd
import numpy as np

a = np.arange(5)
a
```

[1]:
```
array([0, 1, 2, 3, 4])
```

Now we can select the first, the second, and the fifth and last elements of our array using a list of array indices.

[2]:
```
a[[0, 1, 4]]
```

[2]:
```
array([0, 1, 4])
```

Boolean indexing can select the same elements, but by creating a boolean array of the same size as the original array, with elements 0, 1 and 4 set to `True`, all others `False`.

[3]:
```
a[[True, True, False, False, True]]
```

[3]:
```
array([0, 1, 4])
```

Note that this is not how regular lists work in Python. We can do simple indexing as well as slicing (as we learned in the last chapter), but not boolean indexing.

[4]:
```
b = [0, 1, 2, 3, 4]
```

(continues on next page)

(continued from previous page)

```
print("This is the fifth element:", b[4])
try:
    print("Can I get multiple?", b[[0,1,4]])
except Exception as ex:
    print("Doesn't work: ", ex)
```

```
This is the fifth element: 4
Doesn't work:  list indices must be integers or slices, not list
```

4.2 Boolean operators allow us to be very expressive

So now we know how to index our array with a single boolean array. But building that array by hand is a pain, so what you will usually end up doing is applying operations to the original array that return a boolean array themselves.

For example, to select all elements less than 3:

[5]:
```
a[a < 3]
```

[5]:
```
array([0, 1, 2])
```

or all elements that are even (i.e. evenly divisible by 2):

[6]:
```
a[a % 2 == 0]
```

[6]:
```
array([0, 2, 4])
```

And we can combine these using expressions, to AND them (&) or OR them (|). With these operators, we can select the same elements from our first example.

[7]:
```
a[(a < 2) | (a >= 4)]
```

[7]:
```
array([0, 1, 4])
```

Another very helpful operator is the *inverse* or *not* operator, (~). We can easily invert the selection from above. Remember to watch your parentheses!

[8]:
```
a[~((a < 2) | (a >= 4))]
```

[8]:
```
array([2, 3])
```

4.3 In pandas you have to be aware of the index

In pandas, boolean indexing works pretty much like in the NumPy examples above, especially in a `Series`. You pass in a vector of boolean values the same length as the `Series`. Note that this vector doesn't have to have an index if it's just a simple list. However, if you use a pandas `Series`, it will contain an index, so it will need to match. A common method to get a matching index is to apply functions to the original series.

4.3.1 Starting with `Series`

[9]:
```
s = pd.Series(np.arange(5), index=list("abcde"))
s
```

[9]:
```
a    0
b    1
c    2
d    3
e    4
dtype: int64
```

[10]:
```
s[[True, True, False, False, True]]           # this vector is just a list of␣
↪boolean values
```

[10]:
```
a    0
b    1
e    4
dtype: int64
```

[11]:
```
s[np.array([True, True, False, False, True])]  # this vector is a NumPy array of␣
↪boolean values
```

[11]:
```
a    0
b    1
e    4
dtype: int64
```

But, since our index is not the default (i.e. not a `RangeIndex`), if we use another `Series` of the same length, it will not work. It needs a matching index. You'll notice that I purposely gave the example `Series` a non-default index so they wouldn't match. If I'd just created the first one with a default index, it would have worked. But in real life, your index could be timestamps, numbers, strings, or other values.

[12]:
```
try:
    # this Series will have an index that is a RangeIndex, i.e. [0,1,2,3,4]
    s[pd.Series([True, True, False, False, True])]
except Exception as ie:
    print(ie)

# make the same values, but with a matching index
s[pd.Series([True, True, False, False, True], index=list("abcde"))]
```

```
Unalignable boolean Series provided as indexer (index of the boolean Series and of␣
↪the indexed object do not match).
```

[12]:
```
a    0
b    1
e    4
dtype: int64
```

But instead of making a new `Series` like that, we'll just base all of our expressions on our source data `Series` or `DataFrame`, then they'll share an index. This looks much cleaner.

[13]:
```
# just like before with NumPy
s[(s < 2) | (s > 3)]
```

[13]:
```
a    0
b    1
e    4
dtype: int64
```

Make note that you need to surround each expression with parentheses because the Python parser will apply the boolean operators incorrectly. Without parentheses around each expression, for the example above, it would apply it as s < (2 | s) < 3. You'll realize you're forgetting parentheses when you get complaints about the boolean operators being applied to a series. See?

[14]:
```
try:
    s[s < 2 | s > 3]
except ValueError as ve:
    print(ve)
```

```
The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(), a.any() or␣
↪a.all().
```

4.3.2 Moving on to `DataFrames`

We can also do boolean indexing on `DataFrames`. A popular way to create the boolean vector is to use one or more of the columns of the `DataFrame`.

[15]:
```
df = pd.DataFrame({'x': np.arange(5), 'y': np.arange(5, 10)})
df[df['x'] < 3]
```

[15]:
```
   x  y
0  0  5
1  1  6
2  2  7
```

You can also supply multiple conditions, just like before with `Series`. (Remember those parentheses!)

[16]:
```
df[(df['x'] < 3) & (df['y'] > 5)]
```

[16]:
```
   x  y
1  1  6
2  2  7
```

4.3.3 `.loc`, `.iloc`, and `[]`

If you remember from previous chapters, pandas has three primary ways to index the containers. The indexing operator (`[]`) is sort of a hybrid of using the index labels or location based offsets. `.loc` is meant for using the index labels, `.iloc` is for integer based indexing. The good news is that all of them accept boolean arrays, and return subsets of the underlying container.

[17]:
```
mask = (df['x'] < 3) & (df['y'] > 5)
mask
```

[17]:
```
0    False
1     True
2     True
3    False
4    False
dtype: bool
```

[18]:
```
display(df[mask])
display(df.loc[mask])
```

```
   x  y
1  1  6
2  2  7
```

```
   x  y
1  1  6
2  2  7
```

Note that `.iloc` is a little different than the others, if you pass in this mask, you'll get an exception.

[19]:
```
try:
    df.iloc[mask]
except NotImplementedError as nie:
    print(nie)
```

```
iLocation based boolean indexing on an integer type is not available
```

This is by design, `.iloc` is only intended to take positional arguments. However, our mask is a `Series` with an index, so it is rejected. You can still pass in a boolean vector, but just pass in the vector itself without the index.

[20]:
```
print(mask.to_numpy())
df.iloc[mask.to_numpy()]
# or
df.iloc[mask.values]
```

```
[False  True  True False False]
```

[20]:
```
   x  y
1  1  6
2  2  7
```

4.3.4 Examples!

I think one of the most helpful things when thinking about boolean indexing is to see some examples. You are only limited by what you can express by grouping together any combination of expressions on your data. You do this by carefully grouping your boolean expressions and using parentheses wisely. It can also help to break the problem into pieces as you work on it.

As usual, we'll build some data to work with.

[21]:
```
from faker import Faker
Faker.seed(0)

fake = Faker()

size = 1000
df = pd.DataFrame([
    {'year': pd.to_numeric(fake.year()),
     'created': pd.to_datetime(fake.date_between('-30y', '-1y')),
     'phone': fake.phone_number(),
     'first_name': fake.first_name(),
     'last_name': fake.last_name(),
     'city': fake.city(),
     'state': fake.state(),
     'id': f'X-{s:04d}'
    } for s in range(size)])
```

[22]:
```
df.dtypes
```

[22]:
```
year                    int64
created        datetime64[ns]
phone                  object
```

(continues on next page)

(continued from previous page)

```
first_name            object
last_name             object
city                  object
state                 object
id                    object
dtype: object
```

[23]:
```
df.head(3)
```

[23]:
```
   year    created                phone first_name last_name           city  \
0  1996 2017-06-09  (048)764-7593x82421  Christine      King      West Corey
1  2016 2002-11-19    815.659.3877x8408    Sabrina     Davis        Pagetown
2  2003 2019-10-28   351-393-3287x11587     Daniel    Arnold  New Marvinside

            state      id
0      California  X-0000
1  South Carolina  X-0001
2         Indiana  X-0002
```

In terms of examples, there's not really too much complexity to deal with, but here's a few to give you an idea what boolean indexing looks like.

[24]:
```
df[df['first_name'] == 'Tony']  # all people named Tony
```

[24]:
```
     year    created             phone first_name last_name         city  \
314  2016 2002-11-08  924-038-4431x718       Tony    Morris  West Wesley

       state      id
314  Alabama  X-0314
```

[25]:
```
df[(df['year'] >= 2000) & (df['state'] == 'Illinois')] # everyone from Illinois since␣
↪2000
```

[25]:
```
     year    created                   phone first_name last_name  \
221  2005 2014-02-15  001-941-579-5453x08378    Jasmine   Gregory
582  2008 2017-07-22         +1-317-773-0834    Matthew    Walker
604  2000 2018-06-26            028.898.9383      Tracy    Romero
753  2010 2020-02-09     +1-582-158-8566x975    Anthony     Davis
854  2010 1993-01-18        211-547-6463x693       Tina   Johnson
873  2012 2003-01-07         +1-826-711-4489     Hannah      Boyd

                  city     state      id
221        Floreshaven  Illinois  X-0221
582          Toniville  Illinois  X-0582
604       Sanchezville  Illinois  X-0604
753        Delgadoview  Illinois  X-0753
854  North Michaelbury  Illinois  X-0854
873       Andreborough  Illinois  X-0873
```

[26]:
```
# let's get the most popular state for 2020
pop_state = df[df['year'] == 2020].groupby('state').count().sort_values(by='year',␣
↪ascending=False).index[0]

# who is from that state for that year?
df[(df['state'] == pop_state) & (df['year'] == 2020)]
```

[26]:
```
     year    created                 phone first_name last_name  \
291  2020 1995-03-30  001-897-038-6066x988       John   Rowland
```

(continues on next page)

(continued from previous page)

```
334  2020 2011-02-15   (330)352-4308x66853      David    Miller
377  2020 2015-09-04   +1-859-362-9588x271      Jared       Cox

               city      state      id
291      Robertport  Tennessee  X-0291
334  West Sarahberg  Tennessee  X-0334
377       Scotttown  Tennessee  X-0377
```

If we only want to deal with 2020 data, we can just make a new smaller `DataFrame` with that data.

[27]:
```
df = df[df['year'] == 2020]
```

4.3.5 Boolean indexing with `isin`

A helpful method that is often paired with boolean indexing is `Series.isin`. It returns a boolean vector with all rows that match one of the elements in the arguments.

[28]:
```
display(df['state'].head())
display(df['state'].isin(['Hawaii']).head(3))
df[df['state'].isin(['Hawaii', 'California', 'Oregon', 'Washington'])] # Pacific␣
↪Ocean people
```

```
119          Alabama
193    Massachusetts
194           Hawaii
254           Oregon
291        Tennessee
Name: state, dtype: object
```

```
119    False
193    False
194     True
Name: state, dtype: bool
```

[28]:
```
     year    created                  phone first_name last_name  \
194  2020 2007-01-19        +1-912-299-9074       Jason     Brown
254  2020 1992-01-08     161.967.1172x62680         Max   Ramirez
699  2020 2019-08-16   001-569-026-7574x181       Stacy   Garrett
915  2020 2010-12-04           135.781.3249       Kayla   Swanson
919  2020 2019-05-09  001-783-847-3265x50863      Joshua      West

                     city       state      id
194  North Deborahborough       Hawaii  X-0194
254            North Terri      Oregon  X-0254
699              Julieside  Washington  X-0699
915             Robertberg  Washington  X-0915
919          South Kenneth       Hawaii  X-0919
```

I'll wrap it up with a slightly more complicated expression.

[29]:
```
df[
    # people NOT from Pacific Ocean bordering states
    ~(df['state'].isin(['Hawaii', 'California', 'Oregon', 'Washington'])) &
    # and NOT from IL
    (df['state'] != 'Illinois') &
    # created since 2010
```

(continues on next page)

(continued from previous page)

```
    (df['created'] >= '2010-01-01')
    ]
```

[29]:
```
     year    created                  phone first_name last_name  \
119  2020 2020-02-14       828.239.2197x444      Jason     Cohen
334  2020 2011-02-15    (330)352-4308x66853      David    Miller
377  2020 2015-09-04    +1-859-362-9588x271      Jared       Cox
391  2020 2012-05-22          (527)497-5415      Sonya    Snyder
508  2020 2019-08-12       697-525-1983x382      David     Cline
774  2020 2016-06-07        +1-794-123-5303  Alexandra     Klein
879  2020 2018-05-24    +1-500-707-0484x253    Lindsay    Jordan
905  2020 2017-04-01  001-151-557-6424x90780    Jeffrey     Davis

                 city          state      id
119       New Kimland        Alabama  X-0119
334    West Sarahberg      Tennessee  X-0334
377         Scotttown      Tennessee  X-0377
391      Buchananview    Mississippi  X-0391
508       Douglasport  West Virginia  X-0508
774    North Jillfort   Pennsylvania  X-0774
879  East Jonathanport           Iowa  X-0879
905       Molinamouth     New Mexico  X-0905
```

I also find it helpful to sometimes create a variable for storing the mask. So for the above example, instead of having to parse the entire expression when reading the code, it can be helpful to have expressive variable names for the parts of the indexing expression.

[30]:
```
non_bordering = ~(df['state'].isin(['Hawaii', 'California', 'Oregon', 'Washington']))
non_illinois = (df['state'] != 'Illinois')

# more readable maybe?
df[non_bordering & non_illinois & (df['created'] >= '2010-01-01')]
```

[30]:
```
     year    created                  phone first_name last_name  \
119  2020 2020-02-14       828.239.2197x444      Jason     Cohen
334  2020 2011-02-15    (330)352-4308x66853      David    Miller
377  2020 2015-09-04    +1-859-362-9588x271      Jared       Cox
391  2020 2012-05-22          (527)497-5415      Sonya    Snyder
508  2020 2019-08-12       697-525-1983x382      David     Cline
774  2020 2016-06-07        +1-794-123-5303  Alexandra     Klein
879  2020 2018-05-24    +1-500-707-0484x253    Lindsay    Jordan
905  2020 2017-04-01  001-151-557-6424x90780    Jeffrey     Davis

                 city          state      id
119       New Kimland        Alabama  X-0119
334    West Sarahberg      Tennessee  X-0334
377         Scotttown      Tennessee  X-0377
391      Buchananview    Mississippi  X-0391
508       Douglasport  West Virginia  X-0508
774    North Jillfort   Pennsylvania  X-0774
879  East Jonathanport           Iowa  X-0879
905       Molinamouth     New Mexico  X-0905
```

Often when building a complex expression, it can be helpful to build it in pieces, so assigning parts of the mask to variables can make a much more complicated expression easier to read, at the cost of extra variables to deal with. In general, I use variables in the mask if I have to reuse them multiple times, but if only used once, I do the entire expression in place.

Boolean indexing is really quite simple, but powerful. Now we'll look at other ways to index data.

Chapter 5

Selecting by Callable

In the previous chapters, we've focused on the three main methods of selecting data in the two main pandas data structures, `Series` and `DataFrame`.

- The array indexing operator, or `[]`
- The `.loc` selector, for selection using the label on the index
- The `.iloc` selector, for selection using the location

We noted in the last entry in the series that all three can take a boolean vector as indexer to select data from the object. It turns out the indexers support another argument type, a *callable*. If you're not familiar with a callable, the simplest example is just a Python function. It can also be an object with a `___call___` method. We'll cover both examples.

When used for pandas selection by the indexer, the callable needs to take one argument, which will be the pandas object being indexed, and return a result that will select data from the dataset. Why would you want to do this? We'll look at some examples of how this can be useful.

As usual, we'll make some data to work with, in this case some simulated employee data.

[1]:
```
import random

import pandas as pd
import numpy as np

from faker import Faker

random.seed(0)
Faker.seed(0)

fake = Faker()

size = 200

# we'll make two data sets, an hourly and a salary and concat them
df = pd.concat([
    pd.DataFrame([
        {'name': fake.name(),
         'job_title': fake.job(),
         'department': random.choice(['accounting', 'sales', 'distribution',
↪'manufacturing', 'marketing', 'engineering']),
         'annual_salary': random.randrange(30_000, 120_000, step=100),
         'typical_hours': 40,
         'hourly_rate': np.nan,
         'salary_or_hourly': 'Salary',
```

(continues on next page)

(continued from previous page)

```
         'id': f'S-{s:04d}'
        } for s in range(size)]),
    pd.DataFrame([
        {'name': fake.name(),
         'job_title': fake.job(),
         'department': random.choice(['accounting', 'sales', 'distribution',
↪'manufacturing', 'marketing', 'engineering']),
         'annual_salary': np.nan,
         'typical_hours': random.choice([5, 10, 20, 25, 30, 35]),
         'hourly_rate': random.randrange(12, 48, step=2),
         'salary_or_hourly': 'Hourly',
         'id': f'H-{s:04d}'
        } for s in range(size)])
])

df = df.set_index('id', drop=True)
```

[2]:
```
df
```

[2]:
```
                     name                job_title     department  \
id
S-0000        Norma Fisher            Futures trader  manufacturing
S-0001        Katelyn Hull              Immunologist  manufacturing
S-0002        Heather Snow     Engineering geologist   distribution
S-0003        Diane Campos            Water engineer  manufacturing
S-0004      Victoria Patel              Town planner   distribution
...                    ...                       ...            ...
H-0195      Natalie Spears             Lexicographer    engineering
H-0196  Mr. James Price MD          Systems developer          sales
H-0197         Connie Ross   Insurance risk surveyor    engineering
H-0198   Mr. Michael Davis         Investment analyst          sales
H-0199       Travis Hoover  Plant breeder/geneticist      marketing

        annual_salary  typical_hours  hourly_rate salary_or_hourly
id
S-0000       107600.0             40          NaN           Salary
S-0001        34100.0             40          NaN           Salary
S-0002        82300.0             40          NaN           Salary
S-0003        71400.0             40          NaN           Salary
S-0004        78800.0             40          NaN           Salary
...               ...            ...          ...              ...
H-0195            NaN             20         36.0           Hourly
H-0196            NaN             20         38.0           Hourly
H-0197            NaN             35         38.0           Hourly
H-0198            NaN             25         20.0           Hourly
H-0199            NaN             20         20.0           Hourly

[400 rows x 7 columns]
```

[3]:
```
df.dtypes
```

[3]:
```
name                object
job_title           object
department          object
annual_salary      float64
typical_hours        int64
hourly_rate        float64
```

(continues on next page)

(continued from previous page)

```
salary_or_hourly     object
dtype: object
```

[4]:
```
df.describe()
```

[4]:
```
       annual_salary  typical_hours  hourly_rate
count     200.000000     400.000000   200.000000
mean    75475.000000      30.512500    27.520000
std     27934.623516      11.817316    10.071583
min     30300.000000       5.000000    12.000000
25%     49450.000000      25.000000    18.000000
50%     77050.000000      37.500000    28.000000
75%     99750.000000      40.000000    36.000000
max    119700.000000      40.000000    46.000000
```

[5]:
```
df.shape
```

[5]:
```
(400, 7)
```

5.1 The callable takes the container and returns a valid indexer

So we have some data, which is a list of employees, some have a full time salary, some have an hourly salary.

Before we give a few examples using callables, let's clarify what the callable function should do. First, the callable will take one argument, which will be the `DataFrame` or `Series` being indexed. Then it will return a valid value for indexing. This could be any value that we've already discussed in earlier chapters.

So, if we are using the array indexing operator, on a `DataFrame` you'll remember that you can pass in a single column, or a list of columns to select.

[6]:
```
def select_job_titles(df):
    return "job_title"

df[select_job_titles]
```

[6]:
```
id
S-0000              Futures trader
S-0001                Immunologist
S-0002       Engineering geologist
S-0003              Water engineer
S-0004                Town planner
                    ...
H-0195               Lexicographer
H-0196           Systems developer
H-0197     Insurance risk surveyor
H-0198          Investment analyst
H-0199    Plant breeder/geneticist
Name: job_title, Length: 400, dtype: object
```

[7]:
```
def select_job_titles_typical_hours(df):
    return ["job_title", "typical_hours"]

df[select_job_titles_typical_hours].dropna()
```

[7]:
```
                     job_title  typical_hours
id
```

(continues on next page)

(continued from previous page)

```
S-0000            Futures trader             40
S-0001              Immunologist             40
S-0002     Engineering geologist             40
S-0003            Water engineer             40
S-0004              Town planner             40
...                          ...            ...
H-0195             Lexicographer             20
H-0196         Systems developer             20
H-0197   Insurance risk surveyor             35
H-0198       Investment analyst             25
H-0199  Plant breeder/geneticist             20

[400 rows x 2 columns]
```

We can also return a boolean indexer, since that's a valid argument for indexing.

[8]:
```
def select_20_hours_or_less(df):
    return df['typical_hours'] <= 20

df[select_20_hours_or_less].head()
```

[8]:
```
                   name             job_title  department  annual_salary  \
id
H-0000   Jennifer Snyder             Printmaker  accounting            NaN
H-0002  Thomas Alexander       Camera operator   marketing            NaN
H-0003    Kristin Tanner  Nurse, mental health   marketing            NaN
H-0006     Steven Bishop    Engineer, drilling   marketing            NaN
H-0008       Debbie Hall  Commissioning editor  accounting            NaN

        typical_hours  hourly_rate salary_or_hourly
id
H-0000              5         14.0           Hourly
H-0002             20         34.0           Hourly
H-0003              5         42.0           Hourly
H-0006             10         24.0           Hourly
H-0008             10         20.0           Hourly
```

You can also use callables for both the first (row indexer) and second (column indexer) arguments in a `DataFrame`.

[9]:
```
df.loc[lambda df: df['typical_hours'] <= 20, lambda df: ['job_title', 'typical_hours
↪']].head()
```

[9]:
```
                   job_title  typical_hours
id
H-0000             Printmaker              5
H-0002       Camera operator             20
H-0003  Nurse, mental health              5
H-0006    Engineer, drilling             10
H-0008  Commissioning editor             10
```

5.2 Callables might be confusing, but can be really useful

OK, so this all seems kind of unnecessary because you could do this much more directly. Why write a separate function to provide another level of redirection?

It may not be obvious, there there is one use case where it's helpful and is something that I do all the time. Maybe you do as well.

Let's say we want to find departments with an average hourly pay rate below some threshold. Usually you'll do a group by followed by a selector on the resulting groupby `DataFrame`. For example:

[10]:
```
temp = df.groupby('job_title').mean()
temp[temp['hourly_rate'] < 20].head()
```

[10]:
```
                                    annual_salary  typical_hours  \
job_title
Accountant, chartered public finance           NaN           35.0
Administrator, Civil Service                   NaN           35.0
Administrator, arts                            NaN            5.0
Associate Professor                            NaN            5.0
Chief Technology Officer                       NaN           25.0

                                    hourly_rate
job_title
Accountant, chartered public finance         14.0
Administrator, Civil Service                 18.0
Administrator, arts                          18.0
Associate Professor                          18.0
Chief Technology Officer                     18.0
```

But with a callable, you can do this without the temporary `DataFrame` variable.

[11]:
```
df.groupby('job_title').mean().loc[lambda df: df['hourly_rate'] < 20].head()
```

[11]:
```
                                    annual_salary  typical_hours  \
job_title
Accountant, chartered public finance           NaN           35.0
Administrator, Civil Service                   NaN           35.0
Administrator, arts                            NaN            5.0
Associate Professor                            NaN            5.0
Chief Technology Officer                       NaN           25.0

                                    hourly_rate
job_title
Accountant, chartered public finance         14.0
Administrator, Civil Service                 18.0
Administrator, arts                          18.0
Associate Professor                          18.0
Chief Technology Officer                     18.0
```

One thing to note is that there's nothing special about these callables. They still have to return the correct values for the selector you are choosing to use. So for example, you can do this using `loc`:

[12]:
```
df.loc[lambda df: df['department'] == 'engineering'].head()
```

[12]:
```
                      name                         job_title   department  \
id
S-0011          Amy Stark  Environmental education officer  engineering
S-0014          Juan Mann       Sports development officer  engineering
S-0015      Thomas Harris                      Toxicologist  engineering
```

(continues on next page)

(continued from previous page)

```
S-0027  Jacqueline Jackson          Physiological scientist  engineering
S-0031         Laura Myers                      Metallurgist  engineering

        annual_salary  typical_hours  hourly_rate salary_or_hourly
id
S-0011       112900.0             40          NaN           Salary
S-0014        37500.0             40          NaN           Salary
S-0015        63800.0             40          NaN           Salary
S-0027        94000.0             40          NaN           Salary
S-0031        63300.0             40          NaN           Salary
```

But you can't do this, because `.iloc` requires a boolean vector without an index (as we talked about in the chapter on boolean indexing).

[13]:
```
try:
    df.iloc[lambda df: df['department'] == 'engineering']
except ValueError as ve:
    print(ve)


# instead, return just the boolean vector
df.iloc[lambda df: (df['department'] == 'engineering').to_numpy()]
# or
df.iloc[lambda df: (df['department'] == 'engineering').values].head()
```

```
iLocation based boolean indexing cannot use an indexable as a mask
```

[13]:
```
                    name                         job_title   department  \
id
S-0011          Amy Stark  Environmental education officer  engineering
S-0014          Juan Mann       Sports development officer  engineering
S-0015      Thomas Harris                      Toxicologist  engineering
S-0027  Jacqueline Jackson          Physiological scientist  engineering
S-0031         Laura Myers                      Metallurgist  engineering

        annual_salary  typical_hours  hourly_rate salary_or_hourly
id
S-0011       112900.0             40          NaN           Salary
S-0014        37500.0             40          NaN           Salary
S-0015        63800.0             40          NaN           Salary
S-0027        94000.0             40          NaN           Salary
S-0031        63300.0             40          NaN           Salary
```

Also, while I've used the `DataFrame` for all these examples, this works in `Series` as well.

[14]:
```
s = df['annual_salary']

s[lambda s: s < 100000]
```

[14]:
```
id
S-0001    34100.0
S-0002    82300.0
S-0003    71400.0
S-0004    78800.0
S-0005    89700.0
           ...
S-0195    43600.0
```

(continues on next page)

(continued from previous page)

```
S-0196    83300.0
S-0197    96000.0
S-0198    41700.0
S-0199    58500.0
Name: annual_salary, Length: 151, dtype: float64
```

In summary, indexing with a callable allows some flexibity for condensing some code that would otherwise require temporary variables. The thing to remember about the callable is that it will need to return a result that is acceptable in the same place as the callable.

Chapter 6

Selecting via `where` and `mask`

Once the basics of indexing were covered in the first three chapters, more detailed topics on how to select and update data are next. In this chapter, we'll look at selecting using `where` and `mask`.

In the third chapter, we covered the concept of boolean indexing. If you remember, boolean indexing allows us to essentially query our data (either a `DataFrame` or a `Series`) and return only the data that matches the boolean vector we use as our indexer. For example, to select odd values in a `Series` like this:

[1]:
```
import pandas as pd
import numpy as np

s = pd.Series(np.arange(10))
s[s % 2 != 0]
```

[1]:
```
1    1
3    3
5    5
7    7
9    9
dtype: int64
```

6.1 Where is not like other indexers

You'll notice that our result here is only 5 elements even though the original `Series` contains 10 elements. This is the whole point of indexing, selecting the values you want. But what happens if you want the shape of your result to match your original data? In this case, you use `where`. The values that are selected by the where condition are returned, the values that are not selected are set to `NaN`. This way, the value you select has the same shape as your original data.

[2]:
```
s.where(s % 2 != 0)
```

[2]:
```
0    NaN
1    1.0
2    NaN
3    3.0
4    NaN
5    5.0
6    NaN
7    7.0
8    NaN
9    9.0
dtype: float64
```

`where` also accepts an optional argument for what you want the other values to be, if `NaN` is not what you want, with some flexibility. For starters, it can be a scalar or `Series/DataFrame`.

[3]:
```
s.where(s % 2 != 0, -1)  # set the non-matching elements to one value
```

[3]:
```
0   -1
1    1
2   -1
3    3
4   -1
5    5
6   -1
7    7
8   -1
9    9
dtype: int64
```

[4]:
```
s.where(s % 2 != 0, -s)   # set the non-matching elements to the negative value of␣
↪the original series
```

[4]:
```
0    0
1    1
2   -2
3    3
4   -4
5    5
6   -6
7    7
8   -8
9    9
dtype: int64
```

Both the first `condition` argument and the second argument, `other`, can be a callable that accepts the `Series` or `DataFrame` and returns either a scalar or a `Series/DataFrame`. The condition callable should return boolean, the other can return whatever value you want for non selected values. So the above could be expressed (more verbosely) as

[5]:
```
s.where(lambda x: x % 2 != 0, lambda x: x * -1)
```

[5]:
```
0    0
1    1
2   -2
3    3
4   -4
5    5
6   -6
7    7
8   -8
9    9
dtype: int64
```

Using `where` will always return a copy of the existing data. But if you want to modify the original, you can by using the `inplace` argument, similar to many other functions in pandas (like `fillna` or `ffill` and others).

[6]:
```
s.where(s % 2 != 0, -s, inplace=True)
s
```

[6]:
```
0    0
1    1
2   -2
```

(continues on next page)

(continued from previous page)

```
3    3
4   -4
5    5
6   -6
7    7
8   -8
9    9
dtype: int64
```

The `mask` method is just the inverse of `where`. Instead of selecting values based on the condition, it selects values where the condition is `False`. Everthing else is the same as above.

[7]:
```
s.mask(s % 2 != 0, 99)
```

[7]:
```
0     0
1    99
2    -2
3    99
4    -4
5    99
6    -6
7    99
8    -8
9    99
dtype: int64
```

6.2 Updating data

One thing that I noticed in writing this that I had missed before is that `where` is the underlying implementation for boolean indexing with the array indexing operator, i.e. `[]` on a `DataFrame`. So you've already been using `where` even if you didn't know it.

This has implications for updating data. So with a `DataFrame` of random floats around 0,

[8]:
```
df = pd.DataFrame(np.random.random_sample((5, 5)) - .5)
df
```

[8]:
```
          0         1         2         3         4
0 -0.376115 -0.163219  0.275077  0.160295 -0.063975
1 -0.202377  0.111553  0.136514  0.497996 -0.454579
2  0.274466 -0.150671  0.126505  0.330624 -0.073409
3 -0.043345  0.135861 -0.481476 -0.407888  0.106033
4  0.033887  0.391916  0.239830 -0.420775  0.069396
```

[9]:
```
df.where(df < 0) # these two are equivalent
df[df < 0]
```

[9]:
```
          0         1         2         3         4
0 -0.376115 -0.163219       NaN       NaN -0.063975
1 -0.202377       NaN       NaN       NaN -0.454579
2       NaN -0.150671       NaN       NaN -0.073409
3 -0.043345       NaN -0.481476 -0.407888       NaN
4       NaN       NaN       NaN -0.420775       NaN
```

You can also do updates. This is not necessarily that practical for most `DataFrames` I normally work with though. This is because I rarely have a `DataFrame` where I want to update across all the columns like this. But for some

instances that might be useful, so here's an example. We could force all the values to be positive by inverting only the negative values.

[10]:
```
df[df < 0] = -df
df
```

[10]:

	0	1	2	3	4
0	0.376115	0.163219	0.275077	0.160295	0.063975
1	0.202377	0.111553	0.136514	0.497996	0.454579
2	0.274466	0.150671	0.126505	0.330624	0.073409
3	0.043345	0.135861	0.481476	0.407888	0.106033
4	0.033887	0.391916	0.239830	0.420775	0.069396

6.3 NumPy `where`: just a bit more flexible

If you're a NumPy user, you are probably familiar with `np.where`. To use it, you supply a condition and optional `x` and `y` values for `True` and `False` results in the condition. This is a bit different than using `where` in pandas, where the object itself provides data for the `True` result, with an optional `False` result (which defaults to `NaN` if not supplied). So here's how you'd use it to select odd values in our `Series`, and set the even values to 99.

[11]:
```
np.where(s % 2 != 0, s, 99)
```

[11]:
```
array([99,  1, 99,  3, 99,  5, 99,  7, 99,  9])
```

Another way to think about this is that the pandas implementation can be used like the NumPy version, just think of the `self` argument of the `DataFrame` as the `x` argument in NumPy. Here I'm just using what is really meant to be an object method like it's a class method.

[12]:
```
pd.Series.where(cond=s % 2 != 0, self=s, other=99)
```

[12]:
```
0    99
1     1
2    99
3     3
4    99
5     5
6    99
7     7
8    99
9     9
dtype: int64
```

6.4 More examples of using where to make new columns

Let's go back to the data set from the last chapter.

[13]:
```
import random

from faker import Faker

random.seed(0)
Faker.seed(0)

fake = Faker()
```

(continues on next page)

(continued from previous page)

```
size = 200

# we'll make two data sets, an hourly and a salary and concat them
sal = pd.concat([
    pd.DataFrame([
        {'name': fake.name(),
         'job_title': fake.job(),
         'department': random.choice(['accounting', 'sales', 'distribution',
↪'manufacturing', 'marketing', 'engineering']),
         'annual_salary': random.randrange(30_000, 120_000, step=100),
         'typical_hours': 40,
         'hourly_rate': np.nan,
         'salary_or_hourly': 'Salary',
         'id': f'S-{s:04d}'
        } for s in range(size)]),
    pd.DataFrame([
        {'name': fake.name(),
         'job_title': fake.job(),
         'department': random.choice(['accounting', 'sales', 'distribution',
↪'manufacturing', 'marketing', 'engineering']),
         'annual_salary': np.nan,
         'typical_hours': random.choice([5, 10, 20, 25, 30, 35]),
         'hourly_rate': random.randrange(12, 48, step=2),
         'salary_or_hourly': 'Hourly',
         'id': f'H-{s:04d}'
        } for s in range(size)])
])

sal = sal.set_index('id', drop=True)
```

One thing to note about this data set is that that there are a lot of `NaN` values because of the different treatment of salaried and hourly employees. As a result, there's a column for annual salary, and separate columns for typical hours and hourly rates. What if we just want to know what a typical full salary would be for any employee, regardless of their category?

Using `where` is one way we could address this if we wanted a uniform data set. Now we won't apply it to the entire `DataFrame` as the update example above, we'll use it to create one column.

[14]:
```
sal['total_pay'] = sal['annual_salary'].where(sal['salary_or_hourly'] == 'Salary',␣
↪sal['typical_hours'] * sal['hourly_rate'] * 52)
```

Another way to do this, would be to selectively update only rows for hourly workers. But to do this, you end up needing to apply a mask multiple times.

[15]:
```
sal['total_pay2'] = sal['annual_salary']
mask = sal['salary_or_hourly'] != 'Salary'
sal.loc[mask, 'total_pay2'] = sal.loc[mask, 'typical_hours'] * sal.loc[mask, 'hourly_
↪rate'] * 52
```

So using `where` can result in a slightly more simple expression, even if it's a little long.

6.5 Use NumPy `where` and `select` for more complicated updates

There are times where you want to create new columns with some sort of complicated condition on a dataframe that might need to be applied across multiple columns. Using NumPy `where` can be helpful for these situations. For example, we can creating an hourly rate column that calculates an hourly equivalent for the salried employees, but use the existing hourly rate.

[16]:
```
sal['hourly_rate_all'] = np.where(sal['salary_or_hourly'] == 'Salary', sal['annual_
↪salary'] / (52 * 40), sal['hourly_rate'])
```

If you have a much more complex scenario, you can use `np.select`. Think of `np.select` as a where with multiple conditions and multiple choices, as opposed to just one condition with two choices. For example, let's say that the hourly rate for employees in the manufacuring and distribution departments are slightly different because of their shift schedule, so their calculation needs to be different. We could do the calculation in one pass. Note that I chose to use the hourly rate as the default (using the last parameter), but could have just as easily made it a third condition.

[17]:
```
conditions = [
    (sal['salary_or_hourly'] == 'Salary') & (sal['department'].isin(['manufacturing',
↪'distribution'])),
    (sal['salary_or_hourly'] == 'Salary') & (~sal['department'].isin(['manufacturing',
↪ 'distribution'])),
]
choices = [
    sal['annual_salary'] / (26 * 75),
    sal['annual_salary'] / (52 * 40)
]
sal['hourly_rate_all2'] = np.select(conditions, choices, sal['hourly_rate'])

sal.head(8)
```

[17]:

id	name	job_title	department	annual_salary	typical_hours	hourly_rate	salary_or_hourly	total_pay	total_pay2	hourly_rate_all	hourly_rate_all2
S-0000	Norma Fisher	Futures trader	manufacturing	107600.0	40	NaN	Salary	107600.0	107600.0	51.730769	55.179487
S-0001	Katelyn Hull	Immunologist	manufacturing	34100.0	40	NaN	Salary	34100.0	34100.0	16.394231	17.487179
S-0002	Heather Snow	Engineering geologist	distribution	82300.0	40	NaN	Salary	82300.0	82300.0	39.567308	42.205128
S-0003	Diane Campos	Water engineer	manufacturing	71400.0	40	NaN	Salary	71400.0	71400.0	34.326923	36.615385
S-0004	Victoria Patel	Town planner	distribution	78800.0	40	NaN	Salary	78800.0	78800.0		
S-0005	Emily Blair	Lawyer	distribution	89700.0	40	NaN	Salary	89700.0	89700.0		
S-0006	Brian Hamilton	Tree surgeon	sales	81600.0	40	NaN	Salary	81600.0	81600.0		
S-0007	Christine Tran	Probation officer	sales	58800.0	40	NaN	Salary	58800.0	58800.0		

(continues on next page)

(continued from previous page)

```
S-0004        37.884615        40.410256
S-0005        43.125000        46.000000
S-0006        39.230769        39.230769
S-0007        28.269231        28.269231
```

In summary, using pandas' `where` and `mask` methods can be useful ways to select and update data in the same shape as your original data. Using NumPy's `where` and `select` can also be very useful for more complicated scenarios.

Chapter 7

Selection in pandas using query.

Now for something a little different.

So far we have covered concepts about indexing and selecting data in both pandas `Series` and `DataFrame`s using concepts like slicing, boolean indexing, and familiar python concepts like callables. We've also seen how `where`, `mask`, and `np.select` can be used. This chapter will cover the details behind the `query` method and requires a little more background and explanation. In the end, you'll see how it's useful and how it can help speed up your calculations.

In pandas, `DataFrame`s have a `query` method that supports selection using an expression as a string. If you're familiar with relational databases, one way to think about the `query` method is that it is similar to performing a SQL query. But I'm not sure that's the best way to think about it for a couple of reasons. First, it is has very limited support for complex expressions (i.e. don't count on writing queries with complex joins or group by expressions). Second, the purpose of the expression is not just to give you a simple query language for selecting data, but to *speed up* expressions.

That being said, using query does offer a number of advantages, with a few complications, which we will cover.

Let's start with a few examples of what an expression in `query` might look like.

[1]:
```
import pandas as pd
import numpy as np

np.random.seed(0)

# create a sample dataframe of floating points around 0
df = pd.DataFrame(np.random.random((5,5)) - .5, columns=list("abcde"))
```

[2]:
```
df
```

[2]:
```
          a         b         c         d         e
0  0.048814  0.215189  0.102763  0.044883 -0.076345
1  0.145894 -0.062413  0.391773  0.463663 -0.116558
2  0.291725  0.028895  0.068045  0.425597 -0.428964
3 -0.412871 -0.479782  0.332620  0.278157  0.370012
4  0.478618  0.299159 -0.038521  0.280529 -0.381726
```

7.1 Using `query` is still boolean indexing

If you look back to Chapter 4 - Boolean Indexing, you will remember that to select rows in a `DataFrame` using any of the indexing methods, you can pass in a boolean vector the same size as the `DataFrame` index. So, if we want to select all rows in our `DataFrame` where the value in column `b` is negative, we pass in a boolean array created by applying the less than (<) operation to the column.

[3]:
```
df[df["b"] < 0]
```

[3]:
```
          a         b         c         d         e
1  0.145894 -0.062413  0.391773  0.463663 -0.116558
3 -0.412871 -0.479782  0.332620  0.278157  0.370012
```

The `query` method is similar. It takes an expression, which is evaluated in the context of the `DataFrame`, and that expression returns a boolean value used for selection. So we can write the same selection like this:

[4]:
```
df.query("b < 0")
```

[4]:
```
          a         b         c         d         e
1  0.145894 -0.062413  0.391773  0.463663 -0.116558
3 -0.412871 -0.479782  0.332620  0.278157  0.370012
```

More complex expressions can be expressed much simpler. For example, to select rows where column `b` is less than column `c` *and* column `c` is less than column `e`, we need to use a fair amount of parentheses and repeat our `df` variable in order to have it parse properly.

[5]:
```
df[(df["b"] < df["c"]) & (df["c"] < df["e"])]
```

[5]:
```
          a         b        c         d         e
3 -0.412871 -0.479782  0.33262  0.278157  0.370012
```

Using `query`, this can be written in a much more elegant way. For combining and modifying expressions, you can use `&` and `|` and `~`, or `and` and `or` and `not`.

[6]:
```
df.query("(b < c) & (c < e)")
df.query("(b < c) and (c < e)")
```

[6]:
```
          a         b        c         d         e
3 -0.412871 -0.479782  0.33262  0.278157  0.370012
```

Or even better

[7]:
```
df.query("b < c < e")
```

[7]:
```
          a         b        c         d         e
3 -0.412871 -0.479782  0.33262  0.278157  0.370012
```

You can also use your index in the query, by using `index`, or you can rename your index and use that.

[8]:
```
df.query("index > 2")
df.index.name = 'idx'
df.query("idx > 2")
```

[8]:
```
            a         b         c         d         e
idx
3   -0.412871 -0.479782  0.332620  0.278157  0.370012
4    0.478618  0.299159 -0.038521  0.280529 -0.381726
```

7.2 Using query lets you use variables from your session

You can also use variables that are in scope inside your expression by prefixing them with an `@`.

[9]:
```
limit = 0.1
df.query("b > @limit")
```

[9]:
```
            a         b         c         d         e
idx
0    0.048814  0.215189  0.102763  0.044883 -0.076345
4    0.478618  0.299159 -0.038521  0.280529 -0.381726
```

If your columns are not valid Python variable names, you'll have to surround them with backticks (Note you'll need pandas 0.25 or higher for this functionality, and 1.0+ for expanded support).

[10]:
```
df['mean value'] = df.mean(axis=1)
df.query('`mean value` > d')
```

[10]:
```
            a         b         c         d         e  mean value
idx
0    0.048814  0.215189  0.102763  0.044883 -0.076345    0.067061
```

The `in` operator is supported in query, so you can use that as well, either using a variable, or existing columns, or as literals. The `===` and `!=` operators work like `in` and `not in` for lists.

[11]:
```
df['name'] = ['x', 'y', 'z', 'p', 'q']
search = ['x', 'y']
df.query("name in ['x', 'y']")
# or
df.query('name in @search')
# or
df.query('name == @search')
# or
df.query("name == ['x', 'y']")
```

[11]:
```
            a         b         c         d         e  mean value name
idx
0    0.048814  0.215189  0.102763  0.044883 -0.076345    0.067061    x
1    0.145894 -0.062413  0.391773  0.463663 -0.116558    0.164472    y
```

7.3 You can use query to update data as well

These query expressions are handy, but what about updating your `DataFrame`? I know we've mostly talked about selecting data in this book so far, but it turns out that you can also update or add columns by using expressions, but with the `eval` method. Note that `eval` returns a copy of the modified data, so you need to assign back to your variable or use the `inplace` argument to retain the changes.

[12]:
```
df.eval('x = a * b')
```

[12]:
```
            a         b         c         d         e  mean value name  \
idx
0    0.048814  0.215189  0.102763  0.044883 -0.076345    0.067061    x
1    0.145894 -0.062413  0.391773  0.463663 -0.116558    0.164472    y
2    0.291725  0.028895  0.068045  0.425597 -0.428964    0.077059    z
3   -0.412871 -0.479782  0.332620  0.278157  0.370012    0.017627    p
4    0.478618  0.299159 -0.038521  0.280529 -0.381726    0.127612    q
```

(continues on next page)

(continued from previous page)

```
            x
idx
0    0.010504
1   -0.009106
2    0.008429
3    0.198088
4    0.143183
```

Note you can pass in multiple expressions to be evaluated in one go.

[13]:
```
df.eval('''
x = a * b
y = c * d
''')
```

[13]:
```
            a         b         c         d         e  mean value name  \
idx
0    0.048814  0.215189  0.102763  0.044883 -0.076345    0.067061    x
1    0.145894 -0.062413  0.391773  0.463663 -0.116558    0.164472    y
2    0.291725  0.028895  0.068045  0.425597 -0.428964    0.077059    z
3   -0.412871 -0.479782  0.332620  0.278157  0.370012    0.017627    p
4    0.478618  0.299159 -0.038521  0.280529 -0.381726    0.127612    q

            x         y
idx
0    0.010504  0.004612
1   -0.009106  0.181651
2    0.008429  0.028960
3    0.198088  0.092520
4    0.143183 -0.010806
```

7.4 What about updates with multiple values?

It turns out there is a also a higher level pandas `eval` function available for doing more complicated updates involving multiple `DataFrames` or `Series`. I won't get it into it in much detail here, but you can use it for the same sorts of expressions as above.

[14]:
```
s = pd.Series(np.random.random(5))
pd.eval('g = df.a + s', target=df)
```

[14]:
```
            a         b         c         d         e  mean value name  \
idx
0    0.048814  0.215189  0.102763  0.044883 -0.076345    0.067061    x
1    0.145894 -0.062413  0.391773  0.463663 -0.116558    0.164472    y
2    0.291725  0.028895  0.068045  0.425597 -0.428964    0.077059    z
3   -0.412871 -0.479782  0.332620  0.278157  0.370012    0.017627    p
4    0.478618  0.299159 -0.038521  0.280529 -0.381726    0.127612    q

            g
idx
0    0.688735
1    0.289247
2    1.236394
3    0.108978
4    0.893280
```

7.5 query is handy for working with many similar DataFrames

One more nice advantage of using `query` (and `eval`) is that you may have a situation where you have multiple `DataFrames` in your code that share column names where you'd like to use the same expression on them. If you didn't use `query`, you'd have to pass these into a function so you refer to these in local python expressions. So as these expressions get more complex, `query` may make sense for code clarity.

[15]:
```
df2 = pd.DataFrame(np.random.random((5,5)) - .5, columns=list("abcde"))

# without query, need to build expression in python
def find_d_gt_e(d):
    return d[d['d'] > d['e']]

find_d_gt_e(df)
find_d_gt_e(df2)

expr = "d > e" # same expression, re-usable anywhere these columns exist

df.query(expr)
df2.query(expr);
```

7.5.1 The main advantage of query is speed, not convenience

So there are a few advantages to the mini-langage used by `query` and `eval`, resulting in cleaner looking code. But there are still some disadvantages to using a query language that differs from Python. First, since you're passing in a string, there's no syntax checking being done by your editor. You may find this annoying or it may make you slightly less efficient. Also, you have to learn a new set of rules about what expressions are allowed and even possible in this mini language.

What is not entirely obvious here is that under the hood you can install a nice library called `numexpr` (docs, src) that exists to make calculations with large NumPy (and pandas) objects potentially much faster. When you use `query` or `eval`, this expression is passed into `numexpr` and optimized using its bag of tricks. Expected performance improvement can be between .95x and up to 20x, with average performance around 3-4x for typical use cases. You can read details in the docs, but essentially `numexpr` takes vectorized operations and makes them work in chunks that optimize for cache and CPU branch prediction. If your arrays are really large, your cache will not be hit much, and if you break your large arrays into very small pieces, your CPU won't be as efficient.

Make sure you install it in your environment first if it's not there.

```
%pip install numexpr
```

Here's a simple NumPy example from the `numexpr` docs that shows what the improvement can look like.

[16]:
```
import numexpr as ne

a = np.random.rand(int(1e6))
b = np.random.rand(int(1e6))

r1 = %timeit -o 2 * a + 3 * b
r2 = %timeit -o ne.evaluate("2 * a + 3 * b")

print(f"{r1.average / r2.average:.2f}x faster")
```

```
5.39 ms ± 147 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
1.78 ms ± 78.8 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
3.02x faster
```

You can see much more detail in the `numexpr` docs if you're interested, but for a pandas specific example, let's see what the difference is when using numexpr for performing calculations with some `DataFrames` in the size range where we can start to expect an improvement (over 200k rows).

[17]:
```
sd = pd.DataFrame(np.random.random(int(3e7)).reshape(int(1e7), 3))
sd2 = pd.DataFrame(np.random.random(int(3e7)).reshape(int(1e7), 3))

r1 = %timeit -o 3 * sd + 4 * sd2

r2 = %timeit -o pd.eval('3 * sd + 4 * sd2')

print(f"{r1.average / r2.average: .2f}x faster")
```

```
117 ms ± 3.81 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
53.4 ms ± 5.07 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
 2.19x faster
```

Depending on what you are doing, some operations will be sped up even more.

[18]:
```
r1 = %timeit -o 3 * sd + sd2 ** 4

r2 = %timeit -o pd.eval('3 * sd + sd2 ** 4')

print(f"{r1.average / r2.average: .2f}x faster")
```

```
225 ms ± 4.09 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
48.5 ms ± 1.42 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
 4.63x faster
```

When you're working with initial exploration of data, or smaller data sets, using `query` and `eval` will probably not be your go-to options for data selection. But consider these methods for speeding up code that needs to work with large data sets or is run repeatedly, it can make a big difference. It's also useful for cleaning up code with complicated selection criteria.

Chapter 8

Indexing time series data in pandas

Quite often the data that we want to analyze has a time based component. Think about data like daily temperatures or rainfall, stock prices, sales data, student attendance, or events like clicks or views of a web application. There is no shortage of sources of data, and new sources are being added all the time. As a result, most pandas users will need to be familiar with time series data at some point.

So far in this book we've indexed our `DataFrame`s and `Series` using integer values or strings. Dealing with times and dates add some complexity and nuance.

A time series is just a pandas `DataFrame` or `Series` that has a time based index. The values in the time series can be anything else that can be contained in the containers, they are just accessed using date or time values. A time series container can be manipulated in many ways in pandas, but for this chapter I will focus just on the basics of indexing. Knowing how indexing works first is important for data exploration and use of more advanced features.

8.1 DatetimeIndex is just an `Index`, so it supports normal indexing

In pandas, a `DatetimeIndex` is used to provide indexing for pandas `Series` and `DataFrame`s and works just like other `Index` types, but provides special functionality for time series operations. We'll cover the common functionality with other `Index` types first, then talk about the basics of partial string indexing.

One word of warning before we get started. It's important for your index to be sorted, or you may get some strange results.

To show how this standard functionality works, let's create some sample time series data with different time resolutions.

[ ]:

[1]:
```
import pandas as pd
import numpy as np

import datetime

np.random.seed(0) # so we get the same data on future runs

# this is an easy way to create a DatetimeIndex
# both dates are inclusive
d_range = pd.date_range("2021-01-01", "2021-01-20")

# this creates another DatetimeIndex, 10000 minutes long
m_range = pd.date_range("2021-01-01", periods=10000, freq="T")
```

(continues on next page)

(continued from previous page)

```
# daily data in a Series
daily = pd.Series(np.random.rand(len(d_range)), index=d_range)
# minute data in a DataFrame
minute = pd.DataFrame(np.random.rand(len(m_range), 1),
                      columns=["value"],
                      index=m_range)

# time boundaries not on the minute boundary, add some random jitter
mr_range = m_range + pd.Series([pd.Timedelta(microseconds=1_000_000.0 * s)
                                for s in np.random.rand(len(m_range))])
# minute data in a DataFrame, but at a higher resolution
minute2 = pd.DataFrame(np.random.rand(len(mr_range), 1),
                       columns=["value"],
                       index=mr_range)
```

[2]:
```
daily.head()
```

[2]:
```
2021-01-01    0.548814
2021-01-02    0.715189
2021-01-03    0.602763
2021-01-04    0.544883
2021-01-05    0.423655
Freq: D, dtype: float64
```

[3]:
```
minute.head()
```

[3]:
```
                        value
2021-01-01 00:00:00  0.978618
2021-01-01 00:01:00  0.799159
2021-01-01 00:02:00  0.461479
2021-01-01 00:03:00  0.780529
2021-01-01 00:04:00  0.118274
```

[4]:
```
minute2.head()
```

[4]:
```
                               value
2021-01-01 00:00:00.556447  0.192209
2021-01-01 00:01:00.926601  0.442881
2021-01-01 00:02:00.156222  0.466286
2021-01-01 00:03:00.867330  0.009143
2021-01-01 00:04:00.500239  0.257215
```

A `DatetimeIndex` has a resolution that indicates to what level the `Index` is indexing the data. The three indices created above have distinct resolutions. This will have ramifications in how we index later on.

[5]:
```
print("daily:", daily.index.resolution)
print("minute:", minute.index.resolution)
print("randomized minute:", minute2.index.resolution)
```

```
daily: day
minute: minute
randomized minute: microsecond
```

Before we get into some of the "special" ways to index a pandas `Series` or `DataFrame` with a `DatetimeIndex`, let's just look at some of the typical indexing functionality.

8.1.1 Basics

Since the basics of indexing have been covered in earlier chapters, you can refer back to them if any of the details don't make sense. However it's important to realize that a `DatetimeIndex` works just like any other index in pandas, but with extra functionality. (The extra functionality can be more useful and convenient, but just hold tight, those details are next). If you already understand basic indexing from the earlier chapters, you can skim quickly until you get to partial string indexing.

Indexing a `DatetimeIndex` using a `datetime`-like object will use exact indexing.

8.1.2 `getitem` a.k.a the array indexing operator (`[]`)

When using `datetime`-like objects for indexing, we need to match the resolution of the index.

This ends up looking fairly obvious for our daily time series.

[6]:
```
daily[pd.Timestamp("2021-01-01")]
```

[6]:
```
0.5488135039273248
```

[7]:
```
try:
    minute[pd.Timestamp("2021-01-01 00:00:00")]
except KeyError as ke:
    print(f"KeyError was raised: {ke}")
```

```
KeyError was raised: Timestamp('2021-01-01 00:00:00')
```

This `KeyError` is raised because when passing a single argument to the `[]` operator of a `DataFrame` will look for a *column*, not a *row*. We have a single column called `value` in our `DataFrame`, so the code above is looking for a column. Since there isn't a column by that name, there is a `KeyError`. We will use other methods for indexing rows in a `DataFrame`.

8.1.3 `.iloc` indexing

Since the `iloc` indexer is integer offset based, it's pretty clear how it works, not much else to say here. This looks just like it did back in Chapter 2, and it works the same for all resolutions.

[8]:
```
daily.iloc[0]
```

[8]:
```
0.5488135039273248
```

[9]:
```
minute.iloc[-1]
```

[9]:
```
value    0.308168
Name: 2021-01-07 22:39:00, dtype: float64
```

[10]:
```
minute2.iloc[4]
```

[10]:
```
value    0.257215
Name: 2021-01-01 00:04:00.500239, dtype: float64
```

8.1.4 .loc indexing

When using datetime-like objects, you need to have exact matches for single indexing. It's important to realize that when you make datetime or pd.Timestamp objects, all the fields you don't specify explicitly will default to 0.

[11]:
```
jan1 = datetime.datetime(2021, 1, 1)
daily.loc[jan1]
```

[11]:
```
0.5488135039273248
```

[12]:
```
minute.loc[jan1]  # the defaults for hour, minute, second make this work
```

[12]:
```
value    0.978618
Name: 2021-01-01 00:00:00, dtype: float64
```

[13]:
```
try:
    # we don't have that exact time, due to the jitter we used to create this index
    minute2.loc[jan1]
except KeyError as ke:
    print("Missing in index: ", ke)
# but we do have a value on that day
# we could construct it manually to the microsecond if needed
jan1_ms = datetime.datetime(2021, 1, 1, 0, 0, 0, microsecond=minute2.index[0].
↪microsecond)
minute2.loc[jan1_ms]
```

```
Missing in index:  datetime.datetime(2021, 1, 1, 0, 0)
```

[13]:
```
value    0.192209
Name: 2021-01-01 00:00:00.556447, dtype: float64
```

8.1.5 Slicing

Slicing with integers works as expected, you can read more about regular slicing in Chapter 3. But here's a few examples of "regular" slicing, which works with the array indexing operator ([]) or the .iloc indexer.

[14]:
```
daily[0:2] # first two, end is not inclusive
```

[14]:
```
2021-01-01    0.548814
2021-01-02    0.715189
Freq: D, dtype: float64
```

[15]:
```
minute[0:2] # same
```

[15]:
```
                        value
2021-01-01 00:00:00  0.978618
2021-01-01 00:01:00  0.799159
```

[16]:
```
minute2[1:5:2]  # every other
```

[16]:
```
                               value
2021-01-01 00:01:00.926601  0.442881
2021-01-01 00:03:00.867330  0.009143
```

[17]:
```
minute2.iloc[1:5:2] # works with the iloc indexer as well
```

[17]:
```
                               value
2021-01-01 00:01:00.926601  0.442881
2021-01-01 00:03:00.867330  0.009143
```

Slicing with datetime-like objects also works. Note that the end item is inclusive, and the defaults for hours, minutes, seconds, and microseconds will set the cutoff for the randomized data on minute boundaries (in our case).

[18]:
```
daily[datetime.date(2021,1,1):datetime.date(2021, 1,3)] # end is inclusive
```

[18]:
```
2021-01-01    0.548814
2021-01-02    0.715189
2021-01-03    0.602763
Freq: D, dtype: float64
```

[19]:
```
minute[datetime.datetime(2021, 1, 1): datetime.datetime(2021, 1, 1, 0, 2, 0)]
```

[19]:
```
                     value
2021-01-01 00:00:00  0.978618
2021-01-01 00:01:00  0.799159
2021-01-01 00:02:00  0.461479
```

[20]:
```
minute2[datetime.datetime(2021, 1, 1): datetime.datetime(2021, 1, 1, 0, 2, 0)]
```

[20]:
```
                               value
2021-01-01 00:00:00.556447  0.192209
2021-01-01 00:01:00.926601  0.442881
```

This sort of slicing work with `[]` and `.loc`, but not `.iloc`, as expected. Remember, `.iloc` is for integer offset indexing.

[21]:
```
minute2.loc[datetime.datetime(2021, 1, 1): datetime.datetime(2021, 1, 1, 0, 2, 0)]
```

[21]:
```
                               value
2021-01-01 00:00:00.556447  0.192209
2021-01-01 00:01:00.926601  0.442881
```

[22]:
```
try:
    # no! use integers with iloc
    minute2.iloc[datetime.datetime(2021, 1, 1): datetime.datetime(2021, 1, 1, 0, 2,␣
↪0)]
except TypeError as te:
    print(te)
```

```
cannot do positional indexing on DatetimeIndex with these indexers [2021-01-01 00:00:
↪00] of type datetime
```

8.2 DateTimeIndex does special indexing with strings

Now things get really interesting and helpful. When working with time series data, partial string indexing can be very helpful and way less cumbersome than working with `datetime` objects. I know we started with objects, but now you see that for interactive use and exploration, strings are very helpful. You can pass in a string that can be parsed as a full date, and it will work for indexing.

[23]:
```
daily["2021-01-04"]
```

[23]:
```
0.5448831829968969
```

[24]:
```
minute.loc["2021-01-01 00:03:00"]
```

[24]:
```
value    0.780529
Name: 2021-01-01 00:03:00, dtype: float64
```

Strings also work for slicing.

[25]:
```
minute.loc["2021-01-01 00:03:00":"2021-01-01 00:05:00"] # end is inclusive
```

[25]:
```
                        value
2021-01-01 00:03:00  0.780529
2021-01-01 00:04:00  0.118274
2021-01-01 00:05:00  0.639921
```

8.3 Partial String Indexing also works to easily select ranges of dates and times

Partial strings can also be used, so you only need to specify part of the data. This can be useful for pulling out a single year, month, or day from a longer dataset.

[26]:
```
daily["2021"]    # all items match (since they were all in 2021)
daily["2021-01"] # this one as well (and only in January for our data)
```

[26]:
```
2021-01-01    0.548814
2021-01-02    0.715189
2021-01-03    0.602763
2021-01-04    0.544883
2021-01-05    0.423655
2021-01-06    0.645894
2021-01-07    0.437587
2021-01-08    0.891773
2021-01-09    0.963663
2021-01-10    0.383442
2021-01-11    0.791725
2021-01-12    0.528895
2021-01-13    0.568045
2021-01-14    0.925597
2021-01-15    0.071036
2021-01-16    0.087129
2021-01-17    0.020218
2021-01-18    0.832620
2021-01-19    0.778157
2021-01-20    0.870012
Freq: D, dtype: float64
```

You can do this on a `DataFrame` as well.

[27]:
```
minute["2021-01-01"]
```

```
/var/folders/4d/d4x2_mkd4d7gmq0f5cj31wkw0000gn/T/ipykernel_85622/2297314224.py:1:␣
↪FutureWarning: Indexing a DataFrame with a datetimelike index using a single string␣
↪to slice the rows, like `frame[string]`, is deprecated and will be removed in a␣
↪future version. Use `frame.loc[string]` instead.
  minute["2021-01-01"]
```

[27]:
```
                        value
2021-01-01 00:00:00  0.978618
2021-01-01 00:01:00  0.799159
2021-01-01 00:02:00  0.461479
2021-01-01 00:03:00  0.780529
2021-01-01 00:04:00  0.118274
...                       ...
2021-01-01 23:55:00  0.607545
```

(continues on next page)

(continued from previous page)

```
2021-01-01 23:56:00  0.526584
2021-01-01 23:57:00  0.537946
2021-01-01 23:58:00  0.937663
2021-01-01 23:59:00  0.305189

[1440 rows x 1 columns]
```

See that deprecation warning? You should no longer use `[]` for `DataFrame` string indexing (as we saw above, `[]` should be used for column access, not rows). Depending on whether the value is found in the index or not, you may get an error or a warning. Use `.loc` instead so you can avoid the confusion.

[28]:
```
minute2.loc["2021-01-01"]
```

[28]:
```
                             value
2021-01-01 00:00:00.556447  0.192209
2021-01-01 00:01:00.926601  0.442881
2021-01-01 00:02:00.156222  0.466286
2021-01-01 00:03:00.867330  0.009143
2021-01-01 00:04:00.500239  0.257215
...                              ...
2021-01-01 23:55:00.880752  0.028078
2021-01-01 23:56:00.275888  0.464846
2021-01-01 23:57:00.433708  0.209192
2021-01-01 23:58:00.031481  0.461800
2021-01-01 23:59:00.894030  0.286064

[1440 rows x 1 columns]
```

If using string slicing, the end point includes *all times in the day*.

[29]:
```
minute2.loc["2021-01-01":"2021-01-02"]
```

[29]:
```
                             value
2021-01-01 00:00:00.556447  0.192209
2021-01-01 00:01:00.926601  0.442881
2021-01-01 00:02:00.156222  0.466286
2021-01-01 00:03:00.867330  0.009143
2021-01-01 00:04:00.500239  0.257215
...                              ...
2021-01-02 23:55:00.713075  0.875200
2021-01-02 23:56:00.727834  0.544145
2021-01-02 23:57:00.570185  0.139800
2021-01-02 23:58:00.744473  0.269005
2021-01-02 23:59:00.884778  0.532621

[2880 rows x 1 columns]
```

But if we include times, it will include partial periods, cutting off the end right up to the microsecond if it is specified.

[30]:
```
minute2.loc["2021-01-01":"2021-01-02 13:32:01"]
```

[30]:
```
                             value
2021-01-01 00:00:00.556447  0.192209
2021-01-01 00:01:00.926601  0.442881
2021-01-01 00:02:00.156222  0.466286
2021-01-01 00:03:00.867330  0.009143
2021-01-01 00:04:00.500239  0.257215
```

(continues on next page)

(continued from previous page)

```
...                             ...
2021-01-02 13:28:00.365539  0.643137
2021-01-02 13:29:00.935450  0.746866
2021-01-02 13:30:00.289798  0.209199
2021-01-02 13:31:00.968944  0.652414
2021-01-02 13:32:00.852729  0.514134

[2253 rows x 1 columns]
```

8.4 Slicing vs. exact matching

Our three datasets have different resolutions in their index: day, minute, and microsecond respectively. If we pass in a string indexing parameter and the resolution of the string is *less* accurate than the index, it will be treated as a slice. If it's the same or more accurate, it's treated as an exact match. Let's use our microsecond (`minute2`) and minute (`minute`) resolution data examples. Note that every time you get a slice of the `DataFrame`, the value returned is a `DataFrame`. When it's an exact match, it's a `Series`.

[31]:
```
minute2.loc["2021-01-01"]          # slice - the entire day
minute2.loc["2021-01-01 00"]       # slice - the first hour of the day
minute2.loc["2021-01-01 00:00"]    # slice - the first minute of the day
minute2.loc["2021-01-01 00:00:00"] # slice - the first minute and second of the day
```

[31]:
```
                            value
2021-01-01 00:00:00.556447  0.192209
```

[32]:
```
print(str(minute2.index[0]))                          # note the string representation␣
↪include the full microseconds
display(minute2.loc["2021-01-01 00:00:00.556447"])    # exact match (string)
minute2.loc[minute2.index[0]]                         # exact match (datetime)
```

```
2021-01-01 00:00:00.556447
```

```
value    0.192209
Name: 2021-01-01 00:00:00.556447, dtype: float64
```

[32]:
```
value    0.192209
Name: 2021-01-01 00:00:00.556447, dtype: float64
```

[33]:
```
minute.loc["2021-01-01"]       # slice - the entire day
minute.loc["2021-01-01 00"]    # slice - the first hour of the day
minute.loc["2021-01-01 00:00"] # exact match
```

[33]:
```
value    0.978618
Name: 2021-01-01 00:00:00, dtype: float64
```

8.5 `asof` gives the last observed value

One way to deal with the sort of issue where you want the last observation at a timestamp is to use `asof`. Often, when you have data that is either randomized in time or may have missing values, getting the most recent value as of a certain time is preffered. You could do this yourself, but it looks little cleaner to use `asof`.

[34]:
```
minute2.loc[:"2021-01-01 00:00:03"].iloc[-1]
# vs
minute2.asof("2021-01-01 00:00:03")
```

[34]:
```
value    0.192209
Name: 2021-01-01 00:00:03, dtype: float64
```

8.6 `truncate` is similar to slicing

You can also use `truncate` which is sort of like slicing. You specify a value of `before` or `after` (or both) to indicate cutoffs for data. Unlike slicing which includes all values that partially match the date, `truncate` assumes 0 for any unspecified values of the date.

[35]:
```
minute2.truncate(after="2021-01-01 00:00:03")
```

[35]:
```
                              value
2021-01-01 00:00:00.556447  0.192209
```

You can now see that time series data can be indexed a bit differently than other types of `Index` in pandas. Understanding time series slicing will allow you to quickly navigate time series data and quickly move on to more advanced time series analysis.